

JAVA BACKEND

COMPLETE PREPARATION GUIDE

Optimized for 2–3 Years Experience | Java Spring Boot Roles

Topics Covered

Part 1 — Core Java (OOP · Collections · HashMap · Concurrency · JVM · GC)
Part 2 — SQL (Joins · Indexes · Window Functions · Must-Practice Queries)
Part 3 — Spring Boot (DI · Beans · REST · JPA · Transactions · Security)
Part 4 — Microservices (API Gateway · Circuit Breaker · Saga · Communication)
Part 5 — Apache Kafka (Architecture · Producers · Consumers · Real-world)
Part 6 — System Design (Cache · Redis · Load Balancer · CAP Theorem)
Part 7 — AWS Basics (EC2 · S3 · Lambda · ELB)
Part 8 — React Basics (Hooks · State · Props · Lifecycle)
Part 9 — Coding Questions (Strings · Arrays · Two Pointer · Sliding Window)

MUST KNOW | GOOD TO KNOW | RARELY ASKED

Interview Answers · Code Examples · Follow-up Questions · Common Mistakes

TABLE OF CONTENTS

PART 1: CORE JAVA

[1.1 OOP Concepts](#)
[1.2 Access Modifiers](#)
[1.3 Static Keyword](#)
[1.4 Final Keyword](#)
[1.5 Abstract Class vs Interface](#)
[1.6 String vs StringBuilder vs StringBuffer](#)
[1.7 Collections Framework](#)
[1.8 HashMap Internals](#)
[1.9 HashSet Internals](#)
[1.10 equals\(\) and hashCode\(\) Contract](#)
[1.11 Exception Handling](#)
[1.12 JVM, JDK, JRE](#)
[1.13 Memory Management](#)
[1.14 Garbage Collection](#)
[1.15 Multithreading](#)
[1.16 Synchronization](#)
[1.17 Volatile Keyword](#)
[1.18 Executor Framework](#)

PART 2: SQL

[2.1 DDL, DML, DCL, TCL](#)
[2.2 Joins](#)
[2.3 GROUP BY](#)
[2.4 HAVING](#)
[2.5 Subqueries](#)
[2.6 Window Functions](#)
[2.7 Indexes](#)
[2.8 Primary Key and Foreign Key](#)
[2.9 Composite Index](#)
[2.10 Normalization](#)

[2.11 Most Asked SQL Queries \(Must Practice\)](#)

PART 3: SPRING BOOT

[3.1 Dependency Injection \(DI\) and IoC](#)

[3.2 Bean Lifecycle](#)

[3.3 Bean Scopes](#)

[3.4 REST APIs in Spring Boot](#)

[3.5 Spring Boot Request Flow](#)

[3.6 Exception Handling in Spring Boot](#)

[3.7 Spring Data JPA](#)

[3.8 Transactions in Spring](#)

[3.9 Spring Security Basics](#)

[3.10 Validation in Spring Boot](#)

PART 4: MICROSERVICES

[4.1 Why Microservices?](#)

[4.2 API Gateway](#)

[4.3 Service Discovery](#)

[4.4 Circuit Breaker](#)

[4.5 Distributed Transactions](#)

[4.6 Communication Patterns](#)

PART 5: APACHE KAFKA

[5.1 Kafka Architecture](#)

[5.2 Producers](#)

[5.3 Consumers and Consumer Groups](#)

[5.4 Offsets and Partitions](#)

[5.5 Real-World Kafka Examples](#)

PART 6: SYSTEM DESIGN BASICS

[6.1 Caching](#)

[6.2 Redis](#)

[6.3 Load Balancer](#)

[6.4 Scalability](#)

[6.5 Rate Limiting](#)

[6.6 CAP Theorem](#)

PART 7: AWS BASICS

[7.1 EC2 \(Elastic Compute Cloud\)](#)

[7.2 S3 \(Simple Storage Service\)](#)

[7.3 AWS Lambda](#)

[7.4 Elastic Load Balancer \(ELB\)](#)

PART 8: REACT BASICS

[8.1 Components, State, and Props](#)

[8.2 Hooks](#)

[8.3 Component Lifecycle \(Hooks Equivalent\)](#)

PART 9: CODING QUESTIONS

[9.1 String Problems](#)

[9.2 Array Problems](#)

[9.3 HashMap Problems](#)

[9.4 HashSet Problems](#)

[9.5 Prime Number](#)

[9.6 Matrix Problems](#)

[9.7 Two Pointer](#)

[9.8 Sliding Window](#)

PART 1: CORE JAVA

Infosys interviews for 2-3 YOE candidates test Core Java fairly deeply because Spring Boot questions build on top of these fundamentals. Expect 6-10 Core Java questions in a typical 45-60 minute technical round.

1.1 OOP Concepts

MUST KNOW

The four pillars: Encapsulation, Abstraction, Inheritance, Polymorphism. Interviewers rarely ask you to define these in isolation — they ask you to explain them with a real example from your project, or ask you to spot which pillar a piece of code demonstrates.

Encapsulation

Interview Answer: Encapsulation means binding data (fields) and the methods that operate on that data into a single unit (class), and restricting direct access to the internal state using access modifiers. We expose controlled access through getters/setters or business methods.

Production Example: In my Spring Boot project, entity classes like Employee kept fields private and exposed only validated setters — e.g., `setSalary()` would reject negative values — so no other layer could corrupt the object's state directly.

```
public class Employee {  
    private double salary;  
  
    public void setSalary(double salary) {  
        if (salary < 0) throw new IllegalArgumentException("Salary cannot be  
negative");  
        this.salary = salary;  
    }  
    public double getSalary() { return salary; }  
}
```

Abstraction

Interview Answer: Abstraction means hiding implementation details and exposing only the necessary behavior. Achieved via abstract classes and interfaces in Java.

Production Example: I used a `PaymentGateway` interface with multiple implementations (`RazorpayGateway`, `StripeGateway`). The calling service only depends on the interface, not on how each gateway actually talks to its API.

Inheritance

Interview Answer: Inheritance allows a class to acquire properties and behavior of another class using 'extends'. It promotes code reuse but should be used carefully — overuse leads to tight coupling (the classic 'fragile base class' problem).

Key point interviewers probe: Java supports single inheritance for classes (no multiple class inheritance) but multiple inheritance for interfaces.

Polymorphism

Interview Answer: Polymorphism means one interface, many implementations. Two types: compile-time (method overloading) and runtime (method overriding via dynamic dispatch).

```
// Compile-time polymorphism (Overloading)

void process(int a) {}

void process(String a) {}

// Runtime polymorphism (Overriding)

class Notification {

    void send() { System.out.println("Generic notification"); }

}

class EmailNotification extends Notification {

    @Override

    void send() { System.out.println("Email sent"); }

}

// Notification n = new EmailNotification(); n.send(); -> "Email sent" (decided
// at runtime)
```

1.2 Access Modifiers

MUST KNOW

Modifier	Same Class	Same Package / Subclass (diff pkg) / Other
private	Yes	No / No / No
default (no modifier)	Yes	Yes / No / No

Modifier	Same Class	Same Package / Subclass (diff pkg) / Other
protected	Yes	Yes / Yes / No
public	Yes	Yes / Yes / Yes

Interview Answer: Access modifiers control visibility/scope of classes, methods, and fields. `private` is most restrictive, `public` is least. `protected` is unique in that it allows access from a subclass in a different package, but only through inheritance — not via a direct object reference.

1.3 Static Keyword

MUST KNOW

static means the member belongs to the class, not to any instance. It's loaded once when the class is loaded by the JVM and shared across all objects.

- Static variable: shared across all instances, stored in the Metaspace (Java 8+; was PermGen pre-8).
- Static method: can be called without creating an object; cannot access instance (non-static) members directly.
- Static block: runs once when class is loaded, typically used for one-time initialization.
- Static nested class: doesn't hold an implicit reference to the outer class instance (unlike inner class).

```
class Counter {
    static int count = 0;           // shared across all instances

    static { System.out.println("Class loaded"); } // static block

    Counter() { count++; }

    static int getCount() { return count; } // static method
}
```

1.4 Final Keyword

MUST KNOW

Used With	Effect
final variable	Value cannot be reassigned once initialized (constant for primitives; for objects, the reference can't change but the object's internal state can)
final method	Cannot be overridden by subclasses

Used With	Effect
final class	Cannot be extended/subclassed (e.g., String, Integer are final)

```
final List<String> names = new ArrayList<>();
names.add("Raj");           // allowed - modifying object state
// names = new ArrayList<>(); // NOT allowed - reassigning reference
```

1.5 Abstract Class vs Interface

MUST KNOW

This is one of the most frequently asked Core Java questions at Infosys. Be ready to explain the difference AND when you'd choose one over the other in a real design.

Aspect	Abstract Class	Interface
Methods	Can have both abstract and concrete methods	Can have abstract, default, static, and private methods (Java 8+)
Fields	Can have instance variables of any type	Only public static final (constants) — except via default methods' local state
Constructor	Can have a constructor	Cannot have a constructor
Inheritance	Single inheritance only (extends)	A class can implement multiple interfaces
Access modifiers	Members can be private/protected/public	Methods are implicitly public (except private interface methods, Java 9+)
When to use	When classes share common state/behavior (IS-A with shared code)	When unrelated classes need to guarantee a contract/capability (CAN-DO)

Interview Answer: I'd use an abstract class when subclasses share common state and partial implementation — for example, a base 'Vehicle' class with a common 'fuelType' field and a 'startEngine()' method shared by Car and Bike. I'd use an interface when unrelated classes need to guarantee a capability — for example, both a PaymentService and a NotificationService might implement an Auditable interface to support audit logging, even though they're otherwise unrelated.

Production Example: In my Spring Boot project, I defined a

NotificationSender interface with `send(Notification n)` and had **EmailSenderImpl** and **SmsSenderImpl** — that let me inject the right implementation using `@Qualifier/Spring DI` without the calling code knowing which one it's using.

1.6 String vs StringBuilder vs StringBuffer

MUST KNOW

Aspect	String	StringBuilder / StringBuffer
Mutability	Immutable — every modification creates a new object	Mutable — modifies in place
Thread safety	Immutable, so inherently thread-safe	StringBuilder: NOT thread-safe. StringBuffer: thread-safe (synchronized methods)
Performance	Slow for repeated concatenation (creates garbage objects)	StringBuilder faster (no sync overhead); StringBuffer slower than StringBuilder due to synchronization
Storage	String pool (for literals) in heap	Heap, regular object
Use case	Fixed/rarely-changing text	Building strings in loops, single-threaded: StringBuilder; multi-threaded shared buffer: StringBuffer

Interview Answer: String is immutable — any modification like concatenation creates a new object in memory, leaving the old one for garbage collection. **StringBuilder** is mutable and used when you need to build/modify strings frequently, like in a loop, because it avoids creating multiple intermediate objects. **StringBuffer** is the same as **StringBuilder** but with synchronized methods, making it thread-safe but slightly slower. In practice, almost always use **StringBuilder** unless you specifically need thread safety on a shared buffer.


```
// Bad - creates many intermediate String objects in a loop

String result = "";

for (String s : list) { result += s; }    // O(n^2) due to repeated copying


// Good

StringBuilder sb = new StringBuilder();

for (String s : list) { sb.append(s); }

String result = sb.toString();           // O(n)
```

1.7 Collections Framework

MUST KNOW

Expect questions on List vs Set vs Map, ArrayList vs LinkedList, and when to use which. This is asked in nearly every Java interview at this experience level.

Collection	Key Trait	When to Use
ArrayList	Backed by dynamic array, fast random access O(1), slow insert/delete in middle O(n)	Frequent reads, infrequent structural changes
LinkedList	Doubly linked list, fast insert/delete O(1) at ends, slow random access O(n)	Frequent insertions/deletions, queue/deque use cases
HashSet	No duplicates, no order guarantee, backed by HashMap	Fast uniqueness checks, don't care about order
LinkedHashSet	No duplicates, maintains insertion order	Uniqueness + order matters
TreeSet	No duplicates, sorted order (Red-Black tree)	Need sorted unique elements
HashMap	Key-value, no order guarantee, allows one null key	General-purpose fast lookup by key
LinkedHashMap	Key-value, maintains insertion order	Need predictable iteration order (e.g., LRU cache base)
TreeMap	Key-value, sorted by key (Red-Black tree)	Need keys in sorted order, range queries

```

List<Employee> employees = new ArrayList<>();

// Comparable - natural ordering, defined inside the class
class Employee implements Comparable<Employee> {

    public int compareTo(Employee other) { return this.id - other.id; }

}

// Comparator - external, flexible, multiple sort strategies
employees.sort(Comparator.comparing(Employee::getSalary).reversed());

```

1.8 HashMap Internals

MUST KNOW

This is one of THE most commonly asked deep-dive questions in Java interviews — almost guaranteed at Infosys for 2-3 YOE. Be ready to draw/ explain the bucket structure.

Interview Answer: HashMap stores key-value pairs in an array of buckets. When you put a key, Java computes `hashCode()` of the key, then applies an internal hash spreading function (XOR of higher and lower bits) to reduce collisions, and uses $(n-1) \& \text{hash}$ to find the bucket index, where n is the array length (always a power of 2). If multiple keys land in the same bucket (collision), they're stored as a linked list of Node objects in that bucket. Since Java 8, if a single bucket's linked list grows beyond 8 entries (`TREEIFY_THRESHOLD`) and the table has at least 64 buckets, that bucket converts to a Red-Black Tree, improving worst-case lookup from $O(n)$ to $O(\log n)$.

```

// Simplified internal logic

int hash = key.hashCode();

hash = hash ^ (hash >>> 16);    // spread bits to reduce collisions

int index = (n - 1) & hash;      // n = table length (power of 2)

// Node stored at table[index] - linked list, or tree if > 8 nodes & table size
// >= 64

```

Default capacity & load factor: Default initial capacity = 16, default load factor = 0.75. Resize (doubling) triggers when $\text{size} > \text{capacity} * \text{loadFactor}$ (i.e., 12 entries for default 16).

1.9 HashSet Internals

GOOD TO KNOW

Interview Answer: HashSet is internally backed by a HashMap. When you call `set.add(element)`, internally it does `map.put(element, PRESENT)` where `PRESENT` is a static dummy Object constant. The uniqueness guarantee of HashSet directly comes from HashMap's key uniqueness — duplicates are detected via `hashCode() + equals()`.

```
// Simplified real JDK implementation

public class HashSet<E> {

    private transient HashMap<E,Object> map;

    private static final Object PRESENT = new Object();

    public boolean add(E e) {

        return map.put(e, PRESENT) == null;

    }

}
```

1.10 equals() and hashCode() Contract

MUST KNOW

Almost every Java interview tests this — especially in the context of using custom objects as HashMap keys or in a HashSet.

- If two objects are equal (per `equals()`), they **MUST** have the same `hashCode()`.
- If two objects have the same `hashCode()`, they are **NOT** required to be equal (hash collision is allowed).
- If you override `equals()`, you **MUST** override `hashCode()` — otherwise the object breaks in hash-based collections.
- `hashCode()` must be consistent: calling it multiple times on the same unmodified object should return the same value.

```

class Employee {
    private int id;
    private String name;

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Employee)) return false;
        Employee e = (Employee) o;
        return id == e.id && Objects.equals(name, e.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, name);
    }
}

```

Interview Answer: If you override equals() without overriding hashCode(), two 'equal' objects could end up in different buckets in a HashMap, meaning a get() lookup could fail to find an entry you just put — because the default Object.hashCode() (based on memory address) wouldn't match even though equals() says they're logically equal. This breaks the equals-hashCode contract and causes subtle bugs in collections.

1.11 Exception Handling

MUST KNOW

Type	Examples	Key Trait
Checked Exception	IOException, SQLException, ClassNotFoundException	Must be declared (throws) or caught at compile time; represents recoverable conditions
Unchecked Exception (RuntimeException)	NullPointerException, ArrayIndexOutOfBoundsException, IllegalArgumentException	Not checked at compile time; usually programming errors
Error	OutOfMemoryError, StackOverflowError	Serious JVM-level problems; not meant to be caught/recovered from

```

try {
    riskyOperation();
} catch (SpecificException e) {
    log.error("Specific failure", e);
    throw new ServiceException("Failed to process", e); // wrap & rethrow with context
} finally {
    cleanupResources(); // always runs - even if return/exception happens above
}

// try-with-resources (preferred for closing resources like connections/streams)
try (Connection conn = dataSource.getConnection()) {
    // conn auto-closed even on exception
}

```

Interview Answer: Checked exceptions force the caller to handle or declare them — used for recoverable situations like file-not-found, where the caller can reasonably do something about it. Unchecked exceptions represent programming bugs (like NPE) and don't force handling. In my Spring Boot services, I create custom exceptions like `ResourceNotFoundException` extending `RuntimeException` so I don't pollute every method signature with 'throws', and handle them centrally using `@ControllerAdvice` / `@ExceptionHandler`.

1.12 JVM, JDK, JRE

MUST KNOW

Term	Full Form	Contains
JVM	Java Virtual Machine	Executes bytecode; provides memory management, GC, security
JRE	Java Runtime Environment	JVM + core libraries needed to RUN Java applications (no compiler)
JDK	Java Development Kit	JRE + development tools (javac compiler, debugger, javadoc, jar tool)

Interview Answer: JDK is what developers install — it includes the compiler (javac) to convert .java to .class bytecode, plus the JRE to run it. JRE is just the runtime needed to execute Java programs — used on end-user/production machines that only run apps, not develop them. JVM is the engine inside JRE that actually interprets/JIT-compiles bytecode into native machine code and manages memory.

1.13 Memory Management

MUST KNOW

Memory Area	Stores	Notes
Heap	Objects, instance variables	Shared across threads; divided into Young Gen (Eden, S0, S1) and Old Gen
Stack	Method call frames, local variables, references	Per-thread; LIFO; throws StackOverflowError if too deep
Metaspace (Java 8+)	Class metadata, static variables	Replaced PermGen; grows dynamically, less prone to OOM than old PermGen
PC Register	Address of currently executing instruction	Per-thread
Native Method Stack	Native (non-Java, e.g., JNI) method calls	Per-thread

1.14 Garbage Collection

GOOD TO KNOW

Interview Answer: Garbage Collection automatically reclaims heap memory occupied by objects that are no longer reachable from any live thread/reference. Java uses generational GC — most objects are short-lived, so the heap is split into Young Generation (Eden + 2 Survivor spaces) and Old Generation. New objects go to Eden; if they survive a few Minor GC cycles, they get promoted to Old Gen. Old Gen is collected less often via Major/Full GC, which is more expensive.

GC Algorithm	Key Trait
Serial GC	Single-threaded, good for small apps/single-core
Parallel GC	Multi-threaded, throughput-focused, default in older JDKs

GC Algorithm	Key Trait
G1 GC (Garbage First)	Default since Java 9, balances throughput and latency, region-based
ZGC / Shenandoah	Very low pause times, designed for huge heaps (mentioned for awareness, rarely deep-dived for 2-3 YOE)

1.15 Multithreading

MUST KNOW

Multithreading questions at this level usually focus on practical understanding (thread lifecycle, synchronized, volatile, ExecutorService) rather than building custom thread pools from scratch.

Interview Answer: A thread is the smallest unit of execution within a process. Java supports multithreading via the Thread class or implementing Runnable (preferred, since Java doesn't support multiple inheritance and Runnable decouples the task from thread mechanics). In real projects, I rarely create raw Thread objects — I use ExecutorService / thread pools, or in Spring, @Async with a configured TaskExecutor, so thread creation/lifecycle is managed for me.

```
// Preferred: Runnable + ExecutorService over raw Thread

Runnable task = () -> processOrder(orderId);

ExecutorService executor = Executors.newFixedThreadPool(5);

executor.submit(task);

executor.shutdown();


// Spring style

@Async

public CompletableFuture<String> sendEmailAsync(String to) {

    emailService.send(to);

    return CompletableFuture.completedFuture("sent");

}
```

Thread State	Meaning
NEW	Thread created but start() not called
RUNNABLE	Eligible to run / running on CPU
BLOCKED	Waiting to acquire a lock held by another thread
WAITING / TIMED_WAITING	Waiting indefinitely or for a specified time (wait(), join(), sleep())
TERMINATED	Finished execution

1.16 Synchronization

MUST KNOW

Interview Answer: **synchronized** ensures that only one thread can execute a block of code or method on a given object/class at a time, preventing race conditions on shared mutable state. It works by acquiring a monitor lock — either on the object instance (synchronized method/block) or the Class object (static synchronized method).


```

class Counter {
    private int count = 0;

    public synchronized void increment() {    // lock on 'this'
        count++;
    }

    public void incrementBlock() {
        synchronized (this) {                // synchronized block - finer
control
            count++;
        }
    }

    public static synchronized void staticMethod() {    // lock on Counter.class
        // ...
    }
}

```

1.17 Volatile Keyword

GOOD TO KNOW

Interview Answer: volatile guarantees visibility, not atomicity. When a variable is marked volatile, every read goes directly to main memory (not a CPU-cached copy), and every write is immediately flushed to main memory. This ensures that if one thread updates a volatile variable, other threads see the latest value immediately. However, volatile does NOT make compound operations like `count++` atomic — for that, you still need `synchronized` or `AtomicInteger`.

```

private volatile boolean running = true; // common pattern: flag read by
multiple threads

public void stop() { running = false; } // writer thread

public void run() {
    while (running) { doWork(); } // reader thread sees update
    immediately
}

// volatile does NOT help here - count++ is read+modify+write, not atomic
private volatile int count = 0;

public void increment() { count++; } // still a race condition!

```

1.18 Executor Framework

MUST KNOW

Interview Answer: The Executor Framework (`java.util.concurrent`) abstracts thread management — instead of manually creating/destroying threads, you submit tasks to a thread pool that reuses threads, improving performance and giving you control over concurrency limits.

Executor Type	Behavior	Use Case
<code>newFixedThreadPool(n)</code>	Fixed number of threads, queued tasks wait if all busy	Predictable, steady workload (e.g., DB-bound tasks)
<code>newCachedThreadPool()</code>	Creates threads as needed, reuses idle ones, can grow unbounded	Many short-lived async tasks
<code>newSingleThreadExecutor()</code>	Single worker thread, tasks run sequentially	Tasks that must execute in order
<code>newScheduledThreadPool(n)</code>	Supports delayed/periodic task execution	Cron-like or polling tasks

```
ExecutorService executor = Executors.newFixedThreadPool(10);

Future<String> future = executor.submit(() -> fetchDataFromExternalApi());

try {
    String result = future.get(5, TimeUnit.SECONDS); // blocks with timeout
} catch (TimeoutException e) {
    future.cancel(true);
} finally {
    executor.shutdown();
}
```

End of Part I — Core Java

PART 2: SQL

SQL is tested in almost every Infosys backend interview. Expect 3-6 SQL questions — some conceptual, at least one or two requiring you to write a query on the spot. Focus on JOINS, GROUP BY/HAVING, subqueries, indexes, and normalization.

2.1 DDL, DML, DCL, TCL

MUST KNOW

Category	Full Name	Commands
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE, RENAME — defines structure of database objects
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE — manipulates data inside tables
DCL	Data Control Language	GRANT, REVOKE — controls permissions/access rights
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT — manages transactions

Key gotcha: TRUNCATE is DDL (cannot be rolled back in most databases); DELETE is DML (can be rolled back within a transaction). This is a classic interview trick question.

2.2 Joins

MUST KNOW

JOINS are the highest-frequency SQL topic in interviews. You need to write them fluently and explain the difference between each type.

Join Type	Behavior / When to Use
INNER JOIN	Returns only rows where there is a match in BOTH tables. Most common join.
LEFT JOIN (LEFT OUTER JOIN)	Returns ALL rows from the left table + matched rows from right. Unmatched right side is NULL.
RIGHT JOIN (RIGHT OUTER JOIN)	Returns ALL rows from the right table + matched rows from left. Unmatched left side is NULL.
FULL OUTER JOIN	Returns ALL rows from both tables. Unmatched sides are NULL. (Not natively supported in MySQL — use UNION of LEFT and RIGHT JOINS)

Join Type	Behavior / When to Use
CROSS JOIN	Cartesian product — every row of left joined to every row of right. Use carefully.
SELF JOIN	A table joined with itself, using aliases. Common for hierarchical data like employee-manager relationships.

```
-- Schema: employees(id, name, dept_id), departments(dept_id, dept_name)

-- INNER JOIN: only employees WITH a department
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;

-- LEFT JOIN: ALL employees, even those without a department (dept_name = NULL)
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;

-- SELF JOIN: employee and their manager (manager_id references employees.id)
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id;

-- Find employees with NO department (LEFT JOIN + NULL check)
SELECT e.name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

2.3 GROUP BY

MUST KNOW

Interview Answer: GROUP BY groups rows that have the same values in specified columns into aggregated rows. Used with aggregate functions (COUNT, SUM, AVG, MAX, MIN). Every column in SELECT that is NOT inside an aggregate function MUST appear in the GROUP BY clause.

```
-- Number of employees per department
SELECT dept_id, COUNT(*) AS employee_count
FROM employees
GROUP BY dept_id;

-- Average salary per department, ordered by salary desc
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
ORDER BY avg_salary DESC;
```

2.4 HAVING

MUST KNOW

Interview Answer: HAVING filters groups (after GROUP BY) — it's like WHERE but for aggregated results. WHERE filters individual rows BEFORE grouping; HAVING filters groups AFTER aggregation. You use HAVING when the filter involves an aggregate function.

```
-- Departments with more than 5 employees
SELECT dept_id, COUNT(*) AS emp_count
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 5;

-- Wrong: you can't use COUNT(*) in WHERE
-- WHERE COUNT(*) > 5 <-- syntax error
```

2.5 Subqueries

MUST KNOW

Interview Answer: A subquery is a query nested inside another query. It can appear in **SELECT**, **FROM**, **WHERE**, or **HAVING** clauses. Correlated subqueries reference columns from the outer query and re-execute for each row of the outer query (can be slow). Non-correlated subqueries execute once and return a result used by the outer query.

```
-- Non-correlated: employees earning more than average salary

SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Correlated subquery: employees who earn more than their department's average
SELECT name, salary, dept_id
FROM employees e
WHERE salary > (
    SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id
);

-- Subquery as a derived table in FROM

SELECT dept_id, avg_sal
FROM (SELECT dept_id, AVG(salary) AS avg_sal FROM employees GROUP BY dept_id) AS
dept_avg
WHERE avg_sal > 60000;

-- Find 2nd highest salary

SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```



```
-- CTE example: Top 3 earning employees per department

WITH ranked AS (
    SELECT name, salary, dept_id,
           RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
    FROM employees
)

SELECT * FROM ranked WHERE rnk <= 3;
```

2.6 Window Functions

GOOD TO KNOW

Window functions are becoming increasingly common in backend interviews. RANK(), ROW_NUMBER(), DENSE_RANK(), and LAG/LEAD are the most tested.

```
-- ROW_NUMBER: unique sequential number per partition
-- RANK: ties get same rank, next rank skips (1,1,3)
-- DENSE_RANK: ties get same rank, next rank doesn't skip (1,1,2)

SELECT name, salary, dept_id,
       ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS row_num,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS
dense_rnk,
       LAG(salary, 1) OVER (ORDER BY hire_date) AS prev_salary,
       LEAD(salary, 1) OVER (ORDER BY hire_date) AS next_salary
FROM employees;
```

2.7 Indexes

MUST KNOW

Interview Answer: An index is a separate data structure (typically a B-Tree) maintained alongside a table that allows the database to find rows quickly without scanning every row (a 'full table scan'). The tradeoff: indexes speed

up reads (SELECT, WHERE, JOIN, ORDER BY) but slow down writes (INSERT, UPDATE, DELETE) because the index must also be updated.

Index Type	Description
B-Tree Index (default)	Balanced tree, supports equality (=) and range queries (<, >, BETWEEN, LIKE 'prefix%'). Used by default in PostgreSQL/MySQL.
Hash Index	Only supports equality (=). Faster than B-Tree for exact lookup. Not useful for range queries.
Composite Index	Index on multiple columns. Order matters — a composite index on (a, b) can be used for queries filtering on 'a' or 'a AND b', but NOT just 'b' alone (leftmost prefix rule).
Unique Index	Enforces uniqueness constraint on indexed columns. Primary key automatically creates one.
Covering Index	An index that contains ALL columns needed by a query — query can be satisfied entirely from the index without touching the actual table rows (index-only scan).
Partial Index	Index only on a subset of rows (WHERE clause in index definition). Smaller, faster for specific patterns.

Leftmost prefix rule (CRITICAL for composite indexes): For a composite index on (col_a, col_b, col_c): the index is usable for queries filtering on col_a alone, col_a + col_b, or col_a + col_b + col_c. It is NOT usable for queries filtering only on col_b or col_c alone.

```
-- Create an index

CREATE INDEX idx_emp_dept ON employees(dept_id);

CREATE INDEX idx_emp_name_dept ON employees(last_name, dept_id); -- composite


-- Check if index is being used

EXPLAIN SELECT * FROM employees WHERE dept_id = 5;

-- Look for "Index Scan" or "Index Only Scan" vs "Seq Scan" (full table scan)


-- When LIKE uses index vs not:

WHERE name LIKE 'Raj%'      -- uses B-Tree index (prefix search)

WHERE name LIKE '%Raj'      -- does NOT use B-Tree index (leading wildcard)
```

2.8 Primary Key and Foreign Key

MUST KNOW

Constraint	Description
PRIMARY KEY	Uniquely identifies each row in a table. Combines NOT NULL + UNIQUE constraints. A table can have only one PK (though it can be composite). Automatically creates a clustered index in most databases.
FOREIGN KEY	Enforces referential integrity — a value in the child table's FK column must exist in the parent table's PK/unique column (or be NULL). Prevents orphaned records.

```

CREATE TABLE departments (
    dept_id    INT PRIMARY KEY,
    dept_name  VARCHAR(100) NOT NULL
);

CREATE TABLE employees (
    emp_id     INT PRIMARY KEY,
    name       VARCHAR(100),
    dept_id    INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
        ON DELETE SET NULL      -- when dept deleted, set emp.dept_id to NULL
        ON UPDATE CASCADE      -- if dept_id PK changes, update FK automatically
);

```

2.9 Composite Index

GOOD TO KNOW

Interview Answer: A composite index spans multiple columns. Column order in the index definition is critical due to the leftmost prefix rule. Design composite indexes based on your most common query WHERE and ORDER BY patterns.

```

-- Index on (last_name, first_name, dept_id)
CREATE INDEX idx_emp_search ON employees(last_name, first_name, dept_id);

-- This query uses the index (uses first column: last_name)
SELECT * FROM employees WHERE last_name = 'Sharma';

-- This also uses the index (uses first two columns)
SELECT * FROM employees WHERE last_name = 'Sharma' AND first_name = 'Raj';

-- This does NOT use the index (skips first column)
SELECT * FROM employees WHERE first_name = 'Raj';
SELECT * FROM employees WHERE dept_id = 5;

```

2.10 Normalization

MUST KNOW

Interview Answer: Normalization is the process of organizing a database to reduce data redundancy and improve data integrity. It involves decomposing tables into smaller related tables.

Normal Form	Rule	Fixes
1NF	Each column has atomic (indivisible) values; no repeating groups; each row is unique	Multi-valued fields, e.g., storing comma-separated phone numbers in one column
2NF	Must be in 1NF + every non-key attribute is fully functionally dependent on the ENTIRE primary key (no partial dependency). Applies when PK is composite.	e.g., in (order_id, product_id) -> product_name depends only on product_id (partial dep) — split to products table
3NF	Must be in 2NF + no transitive dependencies (non-key column depends on another non-key column)	e.g., employees table: zip_code -> city (zip determines city — transitive dep); move city to a zip_codes table
BCNF	Stricter 3NF — every determinant must be a candidate key	Handles edge cases 3NF misses with overlapping composite keys

Denormalization: Sometimes intentional — trading redundancy for read performance (e.g., adding a redundant 'dept_name' column to employees to avoid a JOIN on every query). Common in analytics/reporting databases and read-heavy microservices.

2.11 Most Asked SQL Queries (Must Practice)

MUST KNOW

These are the most commonly asked write-a-query questions in Infosys interviews. Practice writing them from memory.

Second Highest Salary

```
-- Method 1: Subquery (most common expected answer)

SELECT MAX(salary) FROM employees

WHERE salary < (SELECT MAX(salary) FROM employees);

-- Method 2: LIMIT + OFFSET (MySQL/PostgreSQL)

SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;

-- Method 3: Dense Rank (handles ties properly)

SELECT salary FROM (

    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM
employees

) t WHERE rnk = 2;
```

Nth Highest Salary (generalized)

```
-- Replace 3 with N for Nth highest

SELECT salary FROM (

    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM
employees

) t WHERE rnk = 3;
```

Duplicate Records

```
-- Find duplicates based on name

SELECT name, COUNT(*) AS cnt

FROM employees

GROUP BY name

HAVING COUNT(*) > 1;

-- Delete duplicates, keep one (PostgreSQL using ctid)

DELETE FROM employees

WHERE ctid NOT IN (

    SELECT MIN(ctid) FROM employees GROUP BY name

);
```

Department-wise Max Salary

```

SELECT dept_id, MAX(salary) AS max_salary
FROM employees
GROUP BY dept_id;

-- With department name (JOIN)
SELECT d.dept_name, MAX(e.salary) AS max_salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name;

```

Employees Who Joined in the Last 30 Days

```

SELECT * FROM employees
WHERE hire_date >= CURRENT_DATE - INTERVAL '30 days'; -- PostgreSQL
-- MySQL: WHERE hire_date >= DATE_SUB(NOW(), INTERVAL 30 DAY)

```

Top N Employees Per Department

```

SELECT * FROM (
    SELECT name, salary, dept_id,
           DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
    FROM employees
) ranked
WHERE rnk <= 3;

```

Employees Earning More Than Their Manager

```

-- employees table has: emp_id, name, salary, manager_id
SELECT e.name AS employee, e.salary, m.name AS manager, m.salary AS
manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;

```

Count of Employees Per Department (Including Empty Departments)

```
SELECT d.dept_name, COUNT(e.emp_id) AS emp_count
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

```
-- CASE WHEN salary categorization
SELECT name, salary,
       CASE
           WHEN salary < 30000 THEN 'Junior'
           WHEN salary BETWEEN 30000 AND 70000 THEN 'Mid'
           ELSE 'Senior'
       END AS salary_band
FROM employees;
```

End of Part 2 — SQL

PART 3: SPRING BOOT

Spring Boot is the primary technical focus for Infosys Java Backend roles. Expect 8-12 questions spanning DI, beans, REST APIs, JPA, transactions, and exception handling. Be ready to write code snippets and explain what's happening under the hood.

3.1 Dependency Injection (DI) and IoC

MUST KNOW

Interview Answer: Dependency Injection is a design pattern where an object's dependencies are provided (injected) by an external entity rather than the object creating them itself. In Spring, the IoC (Inversion of Control) container manages object creation and wiring. Instead of 'new MyService()', Spring creates and injects it wherever needed. This promotes loose coupling, testability (you can inject mocks), and follows the Dependency Inversion Principle.

Injection Type	Notes
Constructor Injection (recommended)	Dependencies passed via constructor. Best for required deps — makes the class truly immutable and explicitly shows what it needs. Easier to test and avoids circular deps.
Setter Injection	Dependencies set via setter methods. Good for optional deps that can change after construction. Less favored for required deps.
Field Injection (@Autowired on field)	Spring injects directly into private field via reflection. Easiest to write but CANNOT test without Spring context, hides dependencies, and is generally discouraged in production code by most style guides.

```
// Recommended: Constructor Injection

@Service

public class OrderService {

    private final PaymentService paymentService;

    private final InventoryService inventoryService;

    // @Autowired optional when there's one constructor (Spring 4.3+)

    public OrderService(PaymentService paymentService,

                        InventoryService inventoryService) {

        this.paymentService = paymentService;

        this.inventoryService = inventoryService;

    }

}

// Avoid in production code: Field Injection

@Service

public class OrderService {

    @Autowired

    private PaymentService paymentService; // hard to unit test, hides
dependencies

}
```

3.2 Bean Lifecycle

MUST KNOW

Interview Answer: The Spring IoC container manages the complete lifecycle of a bean: instantiation → dependency injection → initialization → ready for use → destruction. Key hooks: **@PostConstruct** (runs after DI, before bean is ready — for setup like establishing connections, loading config) and **@PreDestroy** (runs before bean is destroyed — for cleanup like closing connections).

```

@Component

public class DatabaseConnectionPool {

    @PostConstruct

    public void init() {

        // Called after all @Autowired dependencies are injected
        // Use for: open connections, load caches, validate config
        log.info("Connection pool initialized");
        pool.initializeConnections(10);

    }

    @PreDestroy

    public void cleanup() {

        // Called before Spring destroys the bean (app shutdown)
        // Use for: close connections, flush buffers, release resources
        pool.closeAll();

    }

}

```

Full lifecycle order: Bean Definition Loaded → Instantiated (no-arg constructor) → @Autowired dependencies injected → BeanNameAware, BeanFactoryAware callbacks → BeanPostProcessor.postProcessBeforeInitialization() → @PostConstruct / afterPropertiesSet() → BeanPostProcessor.postProcessAfterInitialization() → Bean is ready → ... → @PreDestroy / destroy()

3.3 Bean Scopes

MUST KNOW

Scope	Behavior	Use Case
singleton (default)	One instance per Spring IoC container	Stateless services, repositories, utilities
prototype	New instance every time the bean is requested	Stateful beans, objects that should not be shared
request	One instance per HTTP request (web apps only)	Request-scoped data (user context, request tracking)

Scope	Behavior	Use Case
session	One instance per HTTP session (web apps only)	User session data
application	One instance per ServletContext	App-wide shared config (similar to singleton but ServletContext scope)

Important gotcha: Injecting a prototype-scoped bean into a singleton bean defeats prototyping — the singleton gets one prototype instance and keeps it. To get a new prototype instance each time from a singleton, use `ApplicationContext.getBean()` or `@Lookup` method injection.

3.4 REST APIs in Spring Boot

MUST KNOW

Interview Answer: REST (Representational State Transfer) is an architectural style where resources are identified by URLs and manipulated using standard HTTP methods. Spring Boot makes it easy to build REST APIs using `@RestController`, `@RequestMapping`, and related annotations.

```

@RestController // = @Controller + @ResponseBody
@RequestMapping("/api/v1/employees")
public class EmployeeController {

    private final EmployeeService employeeService;

    public EmployeeController(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<EmployeeDTO> getById(@PathVariable Long id) {
        return ResponseEntity.ok(employeeService.findById(id));
    }

    @PostMapping
    public ResponseEntity<EmployeeDTO> create(
        @RequestBody @Valid CreateEmployeeRequest request) {
        EmployeeDTO created = employeeService.create(request);
        URI location = ServletUriComponentsBuilder
            .fromCurrentRequest().path("/{
{id}").buildAndExpand(created.getId()).toUri();
        return ResponseEntity.created(location).body(created); // 201 Created
    }

    @PutMapping("/{id}")
    public ResponseEntity<EmployeeDTO> update(
        @PathVariable Long id,
        @RequestBody @Valid UpdateEmployeeRequest request) {
        return ResponseEntity.ok(employeeService.update(id, request));
    }

    @DeleteMapping("/{id}")

```

Annotation	Purpose
@RestController	@Controller + @ResponseBody — serializes return value to JSON/XML automatically
@RequestMapping	Base URL mapping for the controller or a specific method
@GetMapping / @PostMapping / @PutMapping / @DeleteMapping / @PatchMapping	Method-level shorthand for @RequestMapping(method=...)
@PathVariable	Binds URI template variable to a method parameter
@RequestParam	Binds HTTP query parameter (?name=value) to a method parameter
@RequestBody	Deserializes HTTP request body (JSON) into a Java object
@ResponseStatus	Sets the HTTP response status code for the annotated method
ResponseEntity<T>	Wraps response body + HTTP status + headers for full control over the response

3.5 Spring Boot Request Flow

MUST KNOW

Understanding the request flow is asked both directly ('what happens when a request hits your Spring Boot app?') and indirectly through questions about filters, interceptors, and exception handling.

Full request flow:

1. Client sends HTTP request to embedded Tomcat/Jetty (default: port 8080).
2. DispatcherServlet (the 'Front Controller') receives all requests as the central HTTP handler.
3. Servlet Filters execute (e.g., CORS filter, security filter chain, logging filter) — before the controller.
4. HandlerMapping maps the request URL to the appropriate @Controller method.
5. HandlerInterceptor.preHandle() runs (if registered) — before the controller method.
6. Argument resolvers prepare method parameters (@PathVariable, @RequestBody deserialization, etc.).
7. Controller method executes, returns a value or ResponseEntity.
8. HandlerInterceptor.postHandle() runs — after controller, before view rendering.
9. MessageConverter serializes the return value to JSON (Jackson by default).
10. Response sent back to client. HandlerInterceptor.afterCompletion() runs.

Filter vs Interceptor	Difference
Servlet Filter	Part of Java EE Servlet spec. Operates on raw HttpServletRequest/Response. Runs OUTSIDE Spring's DispatcherServlet. Applied to ALL requests including static resources. Use for cross-cutting concerns: CORS, security, compression, authentication.
Spring HandlerInterceptor	Spring-specific. Runs INSIDE Spring's DispatcherServlet but before/after the controller. Has access to the matched handler (controller method info). Cannot modify the response body after it's committed.

3.6 Exception Handling in Spring Boot

MUST KNOW

Interview Answer: The recommended way to handle exceptions globally in Spring Boot is with `@ControllerAdvice` + `@ExceptionHandler`. This centralizes all error handling in one place instead of try-catching in every controller, resulting in consistent error response format across all APIs.


```
// Custom exception

public class ResourceNotFoundException extends RuntimeException {

    public ResourceNotFoundException(String message) { super(message); }

}

// Global exception handler

@RestControllerAdvice    // = @ControllerAdvice + @ResponseBody

public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)

    public ResponseEntity<ErrorResponse>
handleNotFound(ResourceNotFoundException ex) {

        ErrorResponse error = new ErrorResponse(404, ex.getMessage(),
LocalDateTime.now());

        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);

    }

    @ExceptionHandler(MethodArgumentNotValidException.class)

    public ResponseEntity<ErrorResponse>
handleValidation(MethodArgumentNotValidException ex) {

        String errors = ex.getBindingResult().getFieldErrors()

            .stream().map(e -> e.getField() + ": " + e.getDefaultMessage())

            .collect(Collectors.joining(", "));

        return ResponseEntity.status(HttpStatus.BAD_REQUEST)

            .body(new ErrorResponse(400, errors, LocalDateTime.now()));

    }

    @ExceptionHandler(Exception.class)

    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {

        log.error("Unexpected error", ex);

        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)

            .body(new ErrorResponse(500, "Internal server error",
LocalDateTime.now()));

    }

}
```

3.7 Spring Data JPA

MUST KNOW

Interview Answer: Spring Data JPA abstracts the JPA (Java Persistence API) layer by providing repositories — interfaces that Spring implements at runtime. You define the interface extending `JpaRepository<Entity, ID>` and Spring generates all CRUD operations. For custom queries you use method name conventions (derived queries), `@Query` (JPQL or native SQL), or `@NamedQuery` on the entity.

```

@Entity
@Table(name = "employees")
public class Employee {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @ManyToOne(fetch = FetchType.LAZY)    // LAZY: don't load department unless
    accessed
    @JoinColumn(name = "dept_id")
    private Department department;
}

// Repository – Spring generates all implementation at runtime
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Derived query from method name:

    List<Employee> findByDepartmentIdAndSalaryGreaterThan(Long deptId, Double
salary);

    // Custom JPQL query:

    @Query("SELECT e FROM Employee e WHERE e.name LIKE %:name%")
    List<Employee> searchByName(@Param("name") String name);

    // Native SQL (when you need DB-specific SQL):

    @Query(value = "SELECT * FROM employees WHERE salary > ?1", nativeQuery =
true)
    List<Employee> findHighEarners(Double minSalary);

    // Pagination built-in:

    Page<Employee> findByDeptId(Long deptId, Pageable pageable);
}

```

Interview Answer: JPA fetch strategy: LAZY loading means the related entity is not fetched from DB until you access the field (avoids unnecessary joins). EAGER loading fetches the related entity immediately with the parent. In most cases, prefer LAZY for @OneToMany and @ManyToMany to avoid N+1 query problems. For @ManyToOne and @OneToOne, EAGER can be fine if you always need the related entity.

N+1 Query Problem: If you fetch a list of 100 orders (1 query) and each order has a LAZY-loaded Customer, accessing customer on each order fires 100 additional queries = N+1 problem. Fix: use JOIN FETCH in @Query to load in one query, or @EntityGraph, or pagination with DTO projections.

```
// Fix N+1 with JOIN FETCH

@Query("SELECT o FROM Order o JOIN FETCH o.customer WHERE o.status = :status")

List<Order> findWithCustomers(@Param("status") OrderStatus status);
```

3.8 Transactions in Spring

MUST KNOW

Interview Answer: @Transactional marks a method (or class) to run within a transaction. Spring creates a proxy around the class that starts a transaction before the method, commits on success, and rolls back on unchecked exceptions (RuntimeException/Error) by default. You must specify rollbackFor to roll back on checked exceptions.

```

@Service

public class TransferService {

    @Transactional          // Default: rolls back on RuntimeException

    public void transfer(Long fromId, Long toId, BigDecimal amount) {

        Account from = accountRepo.findById(fromId).orElseThrow();

        Account to = accountRepo.findById(toId).orElseThrow();

        from.debit(amount);          // if this throws RuntimeException, whole
tx rolls back

        to.credit(amount);

        accountRepo.save(from);

        accountRepo.save(to);

    }

    @Transactional(rollbackFor = Exception.class) // Roll back on checked
exceptions too

    public void transferWithChecked(...) throws InsufficientFundsException { ...
}

    @Transactional(readOnly = true) // Hint to DB: read-only, allows
optimizations

    public Employee getEmployee(Long id) { return
repo.findById(id).orElseThrow(); }

}

```

Propagation	Behavior	Use When
REQUIRED (default)	Joins existing transaction if one exists; creates new one if none	Most common — standard transactional service method
REQUIRES_NEW	Always creates a new transaction, suspends existing one	Audit logging that must commit even if outer tx rolls back
NESTED	Creates a savepoint within existing transaction; can roll back to savepoint without rolling back outer tx	Complex partial rollbacks within one logical transaction
NOT_SUPPORTED	Runs without a transaction, suspends existing one	Non-transactional read operations
NEVER	Throws exception if a transaction already exists	Methods that must never be called within a transaction

Propagation	Behavior	Use When
MANDATORY	Requires an existing transaction; throws if none exists	Internal methods that must be called from transactional context

Critical @Transactional gotcha: Self-invocation (calling a @Transactional method from another method in the SAME class) bypasses Spring's proxy — the transaction is NOT started. This is the most common @Transactional bug. Fix: inject 'self' or extract to a separate Spring bean.

3.9 Spring Security Basics

GOOD TO KNOW

Interview Answer: Spring Security provides authentication (who are you?) and authorization (what can you do?) as a filter-chain that sits in front of every request. For modern REST APIs, the common pattern is JWT-based stateless authentication: the client sends credentials once, gets a JWT token, and sends that token in every subsequent request's Authorization header.

```
// Simplified Spring Security 6 config

@Configuration
@EnableWebSecurity

public class SecurityConfig {

    @Bean

    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        return http

            .csrf(csrf -> csrf.disable()) // Disable CSRF for
stateless APIs

            .sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))

            .authorizeHttpRequests(auth -> auth

                .requestMatchers("/api/auth/**").permitAll()

                .requestMatchers("/api/admin/**").hasRole("ADMIN")

                .anyRequest().authenticated()

            )

            .addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class)

            .build();

    }

    @Bean

    public PasswordEncoder passwordEncoder() {

        return new BCryptPasswordEncoder(); // Never store plain-text passwords

    }

}
```

3.10 Validation in Spring Boot

MUST KNOW

Interview Answer: Spring Boot uses Bean Validation (JSR-380, implemented by Hibernate Validator) for request validation. Add **@Valid** or **@Validated** on **@RequestBody** parameters to trigger validation. Use **@ControllerAdvice** to

catch `MethodArgumentNotValidException` and return structured error responses.

```
public class CreateEmployeeRequest {

    @NotBlank(message = "Name is required")

    private String name;

    @Email(message = "Invalid email format")

    private String email;

    @Min(value = 18, message = "Age must be at least 18")

    @Max(value = 65, message = "Age must be at most 65")

    private Integer age;

    @NotNull

    @Positive(message = "Salary must be positive")

    private BigDecimal salary;

    @Pattern(regexp = "[0-9]{10}$", message = "Phone must be 10 digits")

    private String phone;
}

// Controller

@PostMapping

public ResponseEntity<EmployeeDTO> create(@RequestBody @Valid
CreateEmployeeRequest req) {

    // If validation fails, MethodArgumentNotValidException is thrown
    automatically

    return ResponseEntity.ok(employeeService.create(req));
}
```


PART 4: MICROSERVICES

Infosys increasingly asks microservices questions for backend roles. For 2-3 YOE, expect high-level pattern questions rather than deep-dive implementation. Know the WHY, key patterns, and trade-offs.

4.1 Why Microservices?

MUST KNOW

Interview Answer: Microservices break a large application into small, independently deployable services, each owning its own data and business domain. Key benefits: independent deployability (deploy one service without touching others), independent scaling (scale only the bottleneck), technology flexibility (each service can use its best-fit stack), and fault isolation (one service crash doesn't bring down the whole system). Trade-offs: network latency, distributed system complexity, harder debugging, data consistency challenges.

Monolith	Microservices
Single deployable unit	Many independent deployable units
Shared DB	Each service owns its DB (polyglot persistence)
Simple debugging (single process)	Complex distributed tracing needed
Easy transactions (local ACID)	Distributed transactions needed (Saga, 2PC)
Scales as one unit	Scale individual services
Tech stack locked	Each service picks its best stack

4.2 API Gateway

MUST KNOW

Interview Answer: An API Gateway is a single entry point for all client requests in a microservices architecture. It handles cross-cutting concerns like routing, authentication, rate limiting, load balancing, SSL termination, and request aggregation — so individual services don't need to implement these themselves. Examples: Spring Cloud Gateway, Kong, AWS API Gateway, Nginx.

- Routing: forward /orders/* to Order Service, /users/* to User Service
- Authentication: validate JWT once at the gateway, don't repeat auth logic in each service
- Rate limiting: protect services from traffic spikes
- Aggregation: combine responses from multiple services into one client response (BFF pattern)

4.3 Service Discovery

GOOD TO KNOW

Interview Answer: In microservices, service instances start and stop dynamically with changing IP addresses, so you can't hardcode URLs. Service discovery solves this: services register themselves (client-side: Eureka/Consul; server-side: Kubernetes DNS) and clients look up available instances to call. Tools: Netflix Eureka (Spring Cloud), Consul, Kubernetes built-in service discovery (preferred in modern setups).

4.4 Circuit Breaker

MUST KNOW

Interview Answer: A circuit breaker prevents cascading failures in microservices. When Service A calls Service B and B is failing/slow, without protection A's threads pile up waiting, eventually bringing down A as well. The circuit breaker pattern has three states: CLOSED (calls go through normally), OPEN (calls fail fast immediately without hitting the failing service, for a configured time), and HALF-OPEN (trial calls to check if the service has recovered — if successful, goes back to CLOSED). In Spring Boot: Resilience4j.

```

@CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
public PaymentResponse processPayment(PaymentRequest request) {
    return paymentClient.process(request); // might fail
}

// Fallback is called when circuit is OPEN or call fails
public PaymentResponse paymentFallback(PaymentRequest request, Exception e) {
    log.warn("Payment service unavailable, using fallback");
    return PaymentResponse.queued(); // queue for later processing
}

# application.yml
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        slidingWindowSize: 10           # last 10 calls tracked
        failureRateThreshold: 50        # open circuit if 50%+ fail
        waitDurationInOpenState: 10s    # wait 10s before trying again
        permittedNumberOfCallsInHalfOpenState: 3

```

4.5 Distributed Transactions

GOOD TO KNOW

Interview Answer: In microservices, a business operation spanning multiple services (e.g., place order → debit payment → reduce inventory) can't use a single ACID database transaction because each service has its own DB. The Saga pattern solves this: break the transaction into local transactions in each service, and use compensating transactions to undo previous steps if a later step fails.

Saga Type	How It Works
Choreography-based Saga	Each service publishes domain events that trigger the next step. Decentralized, no coordinator. Good for simple flows. Harder to track overall saga state.

Saga Type	How It Works
Orchestration-based Saga	A central orchestrator (Saga Orchestrator) tells each service what to do next and handles compensation on failure. Explicit flow, easier to debug. Introduces a central point.

4.6 Communication Patterns

MUST KNOW

Pattern	Technology	When to Use
Synchronous REST	HTTP/REST, Spring WebClient/RestTemplate/Feign	Simple request-response, immediate result needed, low volume
Synchronous gRPC	Protocol Buffers over HTTP/2	High-performance inter-service calls, polyglot environments, streaming
Async Messaging	Kafka, RabbitMQ, SQS	Decoupling, high throughput, eventual consistency, event-driven, fire-and-forget
Event Sourcing	Kafka + Event Store	Audit trail, replay history, CQRS pattern

```
// Feign Client example
@FeignClient(name = "payment-service", url = "${payment.service.url}")
public interface PaymentServiceClient {
    @PostMapping("/api/payments")
    PaymentResponse processPayment(@RequestBody PaymentRequest request);
}
```

PART 5: APACHE KAFKA

Given your background with Kafka basics, expect 3-5 Kafka questions. Focus on understanding the architecture, partitions, consumer groups, and practical Spring Boot integration.

5.1 Kafka Architecture

MUST KNOW

Interview Answer: Kafka is a distributed event streaming platform. Key components: Broker (server that stores and serves messages), Topic (logical channel categorizing messages), Partition (physical ordered log within a topic — allows parallelism), Producer (publishes messages), Consumer (reads messages), Consumer Group (a group of consumers sharing partitions for parallel processing), ZooKeeper/KRaft (cluster coordination — newer versions use KRaft, removing ZooKeeper dependency).

- Topics are divided into Partitions. Each partition is an ordered, immutable sequence of records.
- Messages in a partition are stored with an incrementing Offset (position in the partition log).
- Kafka retains messages for a configurable period (e.g., 7 days) regardless of whether they've been consumed — unlike traditional queues.
- Within a Consumer Group, each partition is consumed by exactly ONE consumer. More partitions = more consumers can work in parallel.

5.2 Producers

MUST KNOW

Interview Answer: A Kafka producer publishes messages to topics. Key configuration choices: partition selection strategy (by key hash, round-robin, or custom partitioner), acknowledgment mode (acks), and batching.

acks Setting	Behavior	Trade-off
acks=0	Producer doesn't wait for any acknowledgment	Fastest, but messages can be lost if broker crashes
acks=1	Leader broker acknowledges after writing to its log	Good balance — may lose messages if leader fails before replicating
acks=all (or -1)	Leader waits for all in-sync replicas to acknowledge	Slowest, highest durability guarantee — use for critical data

```
// Spring Boot Kafka Producer

@Service

public class OrderEventPublisher {

    private final KafkaTemplate<String, OrderEvent> kafkaTemplate;

    public void publishOrderCreated(Order order) {

        OrderEvent event = new OrderEvent(order.getId(), "ORDER_CREATED",
        LocalDateTime.now());

        kafkaTemplate.send("order-events", order.getId().toString(), event)

            .whenComplete((result, ex) -> {

                if (ex != null) log.error("Failed to send event", ex);

                else log.info("Sent to partition {} offset {}",

                    result.getRecordMetadata().partition(),

                    result.getRecordMetadata().offset());

            });

    }

}
```

5.3 Consumers and Consumer Groups

MUST KNOW

Interview Answer: A consumer reads messages from a topic. A Consumer Group is a logical grouping of consumers that collectively consume all partitions of a topic — each partition is assigned to exactly one consumer in the group at a time. This allows horizontal scaling of message processing: add more consumers to the group (up to the number of partitions) to increase throughput.


```
// Spring Boot Kafka Consumer

@Service

public class OrderEventConsumer {

    @KafkaListener(
        topics = "order-events",
        groupId = "order-processing-group",
        concurrency = "3" // 3 consumer threads = 3
        partitions consumed in parallel
    )
    public void handleOrderEvent(OrderEvent event,
                                @Header(KafkaHeaders.RECEIVED_PARTITION) int
partition,
                                @Header(KafkaHeaders.OFFSET) long offset) {
        log.info("Processing event from partition {} offset {}", partition,
offset);
        orderService.process(event);
    }
}

# application.yml

spring:
  kafka:
    consumer:
      auto-offset-reset: earliest # Start from beginning if no committed
offset
      enable-auto-commit: false # Manual commit for at-least-once
guarantee
      listener:
        ack-mode: MANUAL_IMMEDIATE
```

5.4 Offsets and Partitions

MUST KNOW

Interview Answer: An offset is the unique sequential ID of a message within a partition. Consumers track their position in each partition by committing offsets — either to Kafka's internal `__consumer_offsets` topic (modern) or externally. The partition key determines which partition a message goes to: Kafka hashes the key and uses modulo partitions. Messages with the same key always go to the same partition, guaranteeing order for related events.

```
// Partition selection by key

// Key: orderId "12345" -> hash("12345") % numPartitions = partition number

// All events for orderId 12345 go to same partition -> ordered processing
guaranteed

// How to choose number of partitions?

// Rule of thumb: num_partitions >= max_consumer_instances_in_group

// More partitions = more parallelism, but also more file handles, leader
elections

// Common starting point: 3-6 partitions per topic for moderate throughput
```

5.5 Real-World Kafka Examples

MUST KNOW

Be ready to describe how you'd use Kafka in a real architecture. These answers are highly valued for 2-3 YOE candidates.

- Order Processing: User places order → OrderService publishes 'ORDER_CREATED' event → PaymentService and InventoryService consume in parallel → each publishes their own events ('PAYMENT_PROCESSED', 'INVENTORY_RESERVED') → OrderService consumes both to finalize order.
- Notification Service: Decouple sending emails/SMS from business logic. Any service publishes 'USER_REGISTERED' event → NotificationService consumes and sends email. Multiple consumer groups can consume same event independently.
- Audit Logging: Every service publishes events to 'audit-log' topic → AuditService consumes and persists to Elasticsearch — no coupling between services and the audit system.
- Real-time Analytics: clickstream events from web → Kafka → Kafka Streams/Flink for real-time aggregation → dashboards.
- Change Data Capture (CDC): Debezium monitors DB transaction logs and publishes every DB change to Kafka topics — other services stay in sync without direct DB coupling.

PART 6: SYSTEM DESIGN BASICS

For 2-3 YOE Infosys interviews, system design questions are usually conceptual ('how would you design X?') rather than deep full-system designs. Know these key concepts cold.

6.1 Caching

MUST KNOW

Interview Answer: Caching stores frequently accessed data in fast memory (in-process or external like Redis) to reduce latency and DB load. Key strategies: Cache-Aside (app checks cache first, loads from DB on miss, puts in cache — most common in Spring with `@Cacheable`), Write-Through (write to cache and DB simultaneously), Write-Behind/Write-Back (write to cache first, async write to DB — risky for data loss), Read-Through (cache auto-loads from DB on miss).

```
// Spring Boot caching with @Cacheable

@Service

public class EmployeeService {

    @Cacheable(value = "employees", key = "#id")

    public Employee getById(Long id) {

        // Only called on cache MISS - result is cached for next call

        return employeeRepo.findById(id).orElseThrow();

    }

    @CachePut(value = "employees", key = "#employee.id")

    public Employee update(Employee employee) {

        // Always executes AND updates the cache

        return employeeRepo.save(employee);

    }

    @CacheEvict(value = "employees", key = "#id")

    public void delete(Long id) {

        // Removes the cache entry on delete

        employeeRepo.deleteById(id);

    }

}
```

6.2 Redis

MUST KNOW

Interview Answer: Redis is an in-memory data structure store used as a cache, session store, pub/sub broker, rate limiter, and distributed lock. Key data structures: String, List, Hash, Set, Sorted Set, and Streams. It's single-threaded for command execution (making most operations atomic), supports persistence (RDB snapshots, AOF logs), and clustering for horizontal scaling.

Redis Use Case	How
Caching DB/API results	SET key value EX 3600 (with TTL). Spring: @Cacheable with Redis cache manager
Session storage	Store user session JSON by sessionId key, with TTL for expiry
Rate limiting	INCR key + EXPIRE — atomic counter per user/IP per time window
Distributed lock	SET key value NX PX timeout — set only if Not eXists with expiry
Pub/Sub	PUBLISH channel message / SUBSCRIBE channel — loose coupling between services
Leaderboard	Sorted Set (ZADD) — score-based rankings with ZRANGE, ZREVRANK
Recent activity feed	List with LPUSH + LTRIM — maintain sliding window of recent items

```
// Spring Boot Redis rate limiter example

@Service

public class RateLimiterService {

    private final RedisTemplate<String, String> redisTemplate;

    public boolean isAllowed(String userId) {

        String key = "rate_limit:" + userId + ":" + LocalDate.now();

        Long count = redisTemplate.opsForValue().increment(key);

        if (count == 1) {

            redisTemplate.expire(key, Duration.ofDays(1));

        }

        return count <= 1000; // 1000 requests per user per day

    }

}
```

6.3 Load Balancer

GOOD TO KNOW

Interview Answer: A load balancer distributes incoming requests across multiple instances of a service to prevent any single instance from being overwhelmed. Algorithms: Round Robin, Weighted Round Robin, Least Connections, IP Hash (sticky sessions — same client always goes to same server). Types: L4 (network layer — TCP/UDP routing) and L7 (application layer — HTTP routing with path/header-based rules, SSL termination).

6.4 Scalability

MUST KNOW

Scaling Type	Meaning / When to Use
Vertical Scaling (Scale Up)	Add more CPU/RAM to the same machine. Simple but has hardware limits and creates single point of failure.
Horizontal Scaling (Scale Out)	Add more instances/servers. More complex (stateless required, distributed coordination) but theoretically unlimited and fault-tolerant.

For microservices to scale horizontally, they must be stateless — no in-process state that differs between instances. Sessions must be stored externally (Redis), configs externalized (environment variables/config server), and messages processed idempotently.

6.5 Rate Limiting

GOOD TO KNOW

Interview Answer: Rate limiting restricts how many requests a client can make in a time window. Algorithms: Token Bucket (tokens replenish at rate r , burst allowed up to bucket size), Leaky Bucket (requests processed at constant rate, excess queued/dropped), Fixed Window Counter (count in fixed time windows — has edge-case burst at window boundaries), Sliding Window (more accurate, no burst at window edge). In practice: Redis with INCR + EXPIRE for simple cases; dedicated libraries like Bucket4j or Resilience4j RateLimiter; or handle at API Gateway level.

6.6 CAP Theorem

GOOD TO KNOW

Interview Answer: CAP Theorem states that a distributed system can guarantee at most 2 of 3 properties simultaneously: Consistency (every read gets the most recent write), Availability (every request gets a response — not necessarily the latest data), Partition Tolerance (system continues operating despite network partition between nodes). Since network partitions are unavoidable in real distributed systems, you must choose between CP (prioritize consistency — e.g., ZooKeeper, HBase) and AP (prioritize availability — e.g., Cassandra, DynamoDB with eventual consistency).

System Type	Examples
CP (Consistent + Partition Tolerant)	HBase, ZooKeeper, Etcd — during partition, system may reject requests to maintain consistency
AP (Available + Partition Tolerant)	Cassandra, DynamoDB, CouchDB — during partition, serves potentially stale data but stays available

PART 7: AWS BASICS

AWS questions for 2-3 YOE Java backend roles are usually conceptual — what each service does, when you'd use it, and basic architecture. No deep certifications expected.

7.1 EC2 (Elastic Compute Cloud)

MUST KNOW

Interview Answer: EC2 provides resizable virtual machines (instances) in the cloud. You choose instance type (cpu/memory/network profile), AMI (pre-configured OS image), and storage (EBS volumes). Used for running Spring Boot microservices that need full control over the OS and runtime environment. Key concepts: Security Groups (firewall rules for the instance), Key Pairs (SSH access), Auto Scaling Groups (auto-scale EC2 instances based on demand), Launch Templates/Configurations.

7.2 S3 (Simple Storage Service)

MUST KNOW

Interview Answer: S3 is an object storage service — store and retrieve any amount of data (files, images, backups, logs) via simple HTTP API. Key concepts: Bucket (container for objects), Object (file + metadata + key), Versioning, Lifecycle Policies (auto-transition to cheaper storage classes or auto-delete after N days), Presigned URLs (time-limited URLs allowing direct client upload/download without going through your backend).

```
// Spring Boot S3 file upload using AWS SDK v2

@Service

public class S3StorageService {

    private final S3Client s3Client;

    public String uploadFile(MultipartFile file, String key) throws IOException
    {

        PutObjectRequest request = PutObjectRequest.builder()

            .bucket("my-app-bucket")

            .key(key)

            .contentType(file.getContentType())

            .build();

        s3Client.putObject(request, RequestBody.fromBytes(file.getBytes()));

        return "https://my-app-bucket.s3.amazonaws.com/" + key;

    }

    // Generate a presigned URL for direct browser upload (no backend needed)

    public String generatePresignedUploadUrl(String key) {

        PutObjectPresignRequest presign = PutObjectPresignRequest.builder()

            .signatureDuration(Duration.ofMinutes(15))

            .putObjectRequest(r -> r.bucket("my-app-bucket").key(key))

            .build();

        return presignerClient.presignPutObject(presign).url().toString();

    }

}
```

7.3 AWS Lambda

GOOD TO KNOW

Interview Answer: Lambda is serverless computing — you upload function code and AWS handles all infrastructure, scaling, and execution. You pay only per invocation/duration, not for idle time. Event-driven: triggered by API Gateway, S3 events, SQS/SNS, DynamoDB Streams, etc. Limitations:

cold start latency (first invocation takes longer), 15-minute max timeout, stateless, limited memory (10GB max).

7.4 Elastic Load Balancer (ELB)

GOOD TO KNOW

Interview Answer: AWS provides managed load balancers: ALB (Application Load Balancer) for HTTP/HTTPS traffic with path-based/host-based routing (L7 — ideal for microservices routing), NLB (Network Load Balancer) for ultra-high throughput TCP/UDP (L4), and CLB (Classic — legacy). ALB integrates with ECS/EKS, supports target groups, health checks, SSL termination, and WebSockets.

PART 8: REACT BASICS

For a Java Backend role, React questions are light — expect 2-3 conceptual questions at most. Know the core concepts at a high level so you can demonstrate full-stack awareness.

8.1 Components, State, and Props

MUST KNOW

Interview Answer: React is a UI library that builds UIs as a tree of components. A Component is a function (functional component) or class that returns JSX (UI description). Props are inputs passed from a parent to a child component — read-only, the child cannot modify them. State is internal, mutable data that belongs to a component — when state changes, the component re-renders. Props flow down (parent to child); state is local or lifted up.

```
// Functional Component with props and state
import { useState } from 'react';

// Parent passes data via props
function EmployeeCard({ employee, onDelete }) {
  return (
    <div>
      <h3>{employee.name}</h3>
      <p>{employee.department}</p>
      <button onClick={() => onDelete(employee.id)}>Delete</button>
    </div>
  );
}

// State example
function EmployeeList() {
  const [employees, setEmployees] = useState([]); // useState hook

  const deleteEmployee = (id) => {
    setEmployees(prev => prev.filter(e => e.id !== id));
  };

  return employees.map(emp =>
    <EmployeeCard key={emp.id} employee={emp} onDelete={deleteEmployee} />
  );
}
```

8.2 Hooks

MUST KNOW

Hook	Purpose	Example Use
useState	Adds local state to a functional component	Form inputs, toggles, counters

Hook	Purpose	Example Use
useEffect	Side effects: data fetching, subscriptions, DOM mutations	Fetch API data on mount, subscribe to WebSocket
useContext	Consume a React Context value anywhere in the tree without prop drilling	Theme, user authentication state, language
useRef	Access DOM elements directly or store mutable value that doesn't trigger re-render	Focus an input, store timer ID
useMemo / useCallback	Memoize expensive computed values / callback functions to avoid unnecessary re-computation on re-render	Performance optimization for heavy calculations or passing stable callbacks to child components

```
// useEffect for data fetching
useEffect(() => {
  const fetchData = async () => {
    const response = await fetch('/api/employees');
    const data = await response.json();
    setEmployees(data);
  };
  fetchData();
  return () => { /* cleanup if needed, e.g., cancel request */ };
}, []); // empty dependency array = run once on mount
// [dep1, dep2] = run when dep1 or dep2 changes
```

8.3 Component Lifecycle (Hooks Equivalent)

GOOD TO KNOW

Class Component Lifecycle	Hooks Equivalent
componentDidMount	useEffect(() => { ... }, []) — empty dep array
componentDidUpdate	useEffect(() => { ... }, [dep]) — with dependencies
componentWillUnmount	Cleanup function returned from useEffect
shouldComponentUpdate	React.memo() for functional components — prevents re-render if props unchanged

End of Parts 4-8 (Microservices, Kafka, System Design, AWS, React)

PART 9: CODING QUESTIONS

For 2-3 YOE Infosys interviews, coding questions are usually straightforward — one or two problems from strings, arrays, or HashMap/HashSet. The focus is on clean, readable code with the right data structure choice and explained time/space complexity. Practice writing code on paper or a whiteboard, not just in an IDE.

9.1 String Problems

MUST KNOW

Reverse a String

```
public String reverse(String s) {  
    return new StringBuilder(s).reverse().toString();  
}  
  
// Manual approach (if asked for logic explanation)  
public String reverseManual(String s) {  
    char[] chars = s.toCharArray();  
    int l = 0, r = chars.length - 1;  
    while (l < r) {  
        char temp = chars[l];  
        chars[l++] = chars[r];  
        chars[r--] = temp;  
    }  
    return new String(chars);  
}  
  
// Time: O(n), Space: O(n)
```

Check Palindrome

```

public boolean isPalindrome(String s) {
    // Clean the string first if needed (only alphanumeric, lowercase)
    s = s.toLowerCase().replaceAll("[^a-z0-9]", "");
    int l = 0, r = s.length() - 1;
    while (l < r) {
        if (s.charAt(l) != s.charAt(r)) return false;
        l++; r--;
    }
    return true;
}

// Time: O(n), Space: O(1) after the string cleaning

```

Count Occurrences of Each Character (Frequency Map)

```

public Map<Character, Integer> charFrequency(String s) {
    Map<Character, Integer> freq = new HashMap<>();
    for (char c : s.toCharArray()) {
        freq.merge(c, 1, Integer::sum); // merge is cleaner than getOrDefault
    }
    return freq;
}

// Time: O(n), Space: O(k) where k = distinct characters (at most 26 for lowercase)

```

Check Anagram

```

public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] count = new int[26];
    for (char c : s.toCharArray()) count[c - 'a']++;
    for (char c : t.toCharArray()) {
        if (--count[c - 'a'] < 0) return false;
    }
    return true;
}

// Time: O(n), Space: O(1) (constant 26-char array)

```

First Non-Repeating Character

```

public char firstUniqueChar(String s) {
    // LinkedHashMap preserves insertion order
    Map<Character, Integer> freq = new LinkedHashMap<>();
    for (char c : s.toCharArray()) freq.merge(c, 1, Integer::sum);
    for (Map.Entry<Character, Integer> e : freq.entrySet()) {
        if (e.getValue() == 1) return e.getKey();
    }
    return '_'; // or throw if none found
}

// Time: O(n), Space: O(k)

```

Longest Common Prefix

```
public String longestCommonPrefix(String[] strs) {  
    if (strs.length == 0) return "";  
    String prefix = strs[0];  
    for (int i = 1; i < strs.length; i++) {  
        while (!strs[i].startsWith(prefix)) {  
            prefix = prefix.substring(0, prefix.length() - 1);  
            if (prefix.isEmpty()) return "";  
        }  
    }  
    return prefix;  
}  
  
// Time: O(S) where S = total characters in all strings, Space: O(1)
```

9.2 Array Problems

MUST KNOW

Find Maximum and Minimum

```
public int[] findMinMax(int[] arr) {  
    int min = arr[0], max = arr[0];  
    for (int num : arr) {  
        min = Math.min(min, num);  
        max = Math.max(max, num);  
    }  
    return new int[]{min, max};  
}  
  
// Time: O(n), Space: O(1)
```

Find Duplicates in Array

```

public List<Integer> findDuplicates(int[] arr) {
    Set<Integer> seen = new HashSet<>();
    List<Integer> duplicates = new ArrayList<>();
    for (int num : arr) {
        if (!seen.add(num)) duplicates.add(num); // add returns false if
already present
    }
    return duplicates;
}
// Time: O(n), Space: O(n)

```

Rotate Array by K Positions

```

public void rotate(int[] nums, int k) {
    k = k % nums.length; // handle k > length
    reverse(nums, 0, nums.length - 1);
    reverse(nums, 0, k - 1);
    reverse(nums, k, nums.length - 1);
}

private void reverse(int[] nums, int l, int r) {
    while (l < r) {
        int temp = nums[l]; nums[l++] = nums[r]; nums[r--] = temp;
    }
}
// Time: O(n), Space: O(1) – in-place using reverse trick

```

Move Zeros to End

```

public void moveZeroes(int[] nums) {
    int insertPos = 0;
    for (int num : nums) {
        if (num != 0) nums[insertPos++] = num;
    }
    while (insertPos < nums.length) nums[insertPos++] = 0;
}
// Time: O(n), Space: O(1)

```

Maximum Subarray (Kadane's Algorithm)

```

public int maxSubArray(int[] nums) {
    int maxSum = nums[0], currentSum = nums[0];
    for (int i = 1; i < nums.length; i++) {
        currentSum = Math.max(nums[i], currentSum + nums[i]); // extend or
restart
        maxSum = Math.max(maxSum, currentSum);
    }
    return maxSum;
}
// Time: O(n), Space: O(1)
// Classic follow-up: "Also return the start and end indices of the subarray"

```

9.3 HashMap Problems

MUST KNOW

Two Sum


```

public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>(); // value -> index
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (seen.containsKey(complement)) {
            return new int[]{seen.get(complement), i};
        }
        seen.put(nums[i], i);
    }
    return new int[]{};
}

// Time: O(n), Space: O(n)
// Better than O(n^2) brute force – the HashMap gives us O(1) lookup

```

Group Anagrams Together

```

public List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strs) {
        char[] arr = s.toCharArray();
        Arrays.sort(arr);
        String key = new String(arr); // sorted string is the same for all
anagrams
        map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
    }
    return new ArrayList<>(map.values());
}

// Time: O(n * k log k) where k = max string length, Space: O(n*k)

```

Longest Substring Without Repeating Characters

```

public int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> charIndex = new HashMap<>();
    int maxLen = 0, left = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (charIndex.containsKey(c) && charIndex.get(c) >= left) {
            left = charIndex.get(c) + 1; // shrink window past the duplicate
        }
        charIndex.put(c, right);
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}

// Time: O(n), Space: O(min(m, n)) where m = charset size
// This IS a sliding window problem – also classified under Section 9.7

```

9.4 HashSet Problems

MUST KNOW

Contains Duplicate

```

public boolean containsDuplicate(int[] nums) {
    Set<Integer> seen = new HashSet<>();
    for (int num : nums) {
        if (!seen.add(num)) return true;
    }
    return false;
}

// Time: O(n), Space: O(n)

```

Intersection of Two Arrays

```

public int[] intersection(int[] nums1, int[] nums2) {
    Set<Integer> set1 = new HashSet<>();
    for (int n : nums1) set1.add(n);

    Set<Integer> result = new HashSet<>();
    for (int n : nums2) {
        if (set1.contains(n)) result.add(n);
    }
    return result.stream().mapToInt(Integer::intValue).toArray();
}
// Time: O(m + n), Space: O(m)

```

Longest Consecutive Sequence

```

public int longestConsecutive(int[] nums) {
    Set<Integer> numSet = new HashSet<>();
    for (int n : nums) numSet.add(n);

    int maxLen = 0;
    for (int n : numSet) {
        if (!numSet.contains(n - 1)) { // start of a sequence
            int len = 1;
            while (numSet.contains(n + len)) len++;
            maxLen = Math.max(maxLen, len);
        }
    }
    return maxLen;
}
// Time: O(n), Space: O(n) – linear despite nested while because each element
visited once

```

9.5 Prime Number

GOOD TO KNOW

Check if a Number is Prime

```
public boolean isPrime(int n) {
    if (n <= 1) return false;
    if (n <= 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    // Check divisors of form 6k ± 1 up to sqrt(n)
    for (int i = 5; (long) i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return false;
    }
    return true;
}

// Time: O(sqrt(n)), Space: O(1)
```

Sieve of Eratosthenes — All Primes Up to N

```
public List<Integer> sievePrimes(int n) {
    boolean[] isComposite = new boolean[n + 1];
    List<Integer> primes = new ArrayList<>();
    for (int i = 2; i <= n; i++) {
        if (!isComposite[i]) {
            primes.add(i);
            for (long j = (long) i * i; j <= n; j += i) {
                isComposite[(int) j] = true;
            }
        }
    }
    return primes;
}

// Time: O(n log log n), Space: O(n)
```

9.6 Matrix Problems

GOOD TO KNOW

Spiral Order Traversal

```

public List<Integer> spiralOrder(int[][] matrix) {
    List<Integer> result = new ArrayList<>();
    if (matrix.length == 0) return result;
    int top = 0, bottom = matrix.length - 1;
    int left = 0, right = matrix[0].length - 1;

    while (top <= bottom && left <= right) {
        for (int i = left; i <= right; i++) result.add(matrix[top][i]);
        top++;
        for (int i = top; i <= bottom; i++) result.add(matrix[i][right]);
        right--;
        if (top <= bottom) {
            for (int i = right; i >= left; i--) result.add(matrix[bottom][i]);
            bottom--;
        }
        if (left <= right) {
            for (int i = bottom; i >= top; i--) result.add(matrix[i][left]);
            left++;
        }
    }
    return result;
}

// Time: O(m*n), Space: O(1) extra

```

Transpose a Matrix

```

public void transpose(int[][] matrix) {
    int n = matrix.length;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) { // only upper triangle
            int temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    }
}

// Time: O(n^2), Space: O(1) in-place

```

Rotate Matrix 90 Degrees Clockwise

```

public void rotate(int[][] matrix) {
    // Step 1: Transpose
    int n = matrix.length;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) {
            int temp = matrix[i][j]; matrix[i][j] = matrix[j][i]; matrix[j][i] =
temp;
        }

    // Step 2: Reverse each row
    for (int i = 0; i < n; i++) {
        int l = 0, r = n - 1;
        while (l < r) {
            int temp = matrix[i][l]; matrix[i][l++] = matrix[i][r]; matrix[i]
[r--] = temp;
        }
    }
}

// Time: O(n^2), Space: O(1) in-place

```

9.7 Two Pointer

MUST KNOW

Two-pointer technique uses two pointers on a sorted array or linked list to avoid nested loops (reducing $O(n^2)$ to $O(n)$).

Two Sum in Sorted Array

```
public int[] twoSumSorted(int[] nums, int target) {  
    int l = 0, r = nums.length - 1;  
    while (l < r) {  
        int sum = nums[l] + nums[r];  
        if (sum == target) return new int[]{l + 1, r + 1};  
        else if (sum < target) l++;  
        else r--;  
    }  
    return new int[]{};  
}  
  
// Time: O(n), Space: O(1)
```

Remove Duplicates from Sorted Array (In-Place)

```
public int removeDuplicates(int[] nums) {  
    if (nums.length == 0) return 0;  
    int slow = 0;  
    for (int fast = 1; fast < nums.length; fast++) {  
        if (nums[fast] != nums[slow]) {  
            nums[++slow] = nums[fast];  
        }  
    }  
    return slow + 1; // new length  
}  
  
// Time: O(n), Space: O(1)
```

Three Sum (Triplets that Sum to Zero)

```

public List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> result = new ArrayList<>();
    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue; // skip duplicates
        int l = i + 1, r = nums.length - 1;
        while (l < r) {
            int sum = nums[i] + nums[l] + nums[r];
            if (sum == 0) {
                result.add(Arrays.asList(nums[i], nums[l], nums[r]));
                while (l < r && nums[l] == nums[l + 1]) l++;
                while (l < r && nums[r] == nums[r - 1]) r--;
                l++; r--;
            } else if (sum < 0) l++;
            else r--;
        }
    }
    return result;
}

// Time: O(n^2), Space: O(1) excluding output

```

9.8 Sliding Window

MUST KNOW

Sliding window maintains a window of elements and slides it across the data. Fixed-size window for max/min in subarray of size k; variable-size window when window boundary depends on a condition.

Maximum Sum of Subarray of Size K (Fixed Window)


```

public int maxSumSubarray(int[] arr, int k) {
    int windowSum = 0, maxSum = 0;
    for (int i = 0; i < k; i++) windowSum += arr[i]; // build initial window
    maxSum = windowSum;
    for (int i = k; i < arr.length; i++) {
        windowSum += arr[i] - arr[i - k]; // slide: add new, remove oldest
        maxSum = Math.max(maxSum, windowSum);
    }
    return maxSum;
}
// Time: O(n), Space: O(1)
// O(n^2) brute force recalculates window sum each time — avoid that

```

Minimum Size Subarray with Sum >= Target (Variable Window)

```

public int minSubArrayLen(int target, int[] nums) {
    int l = 0, sum = 0, minLen = Integer.MAX_VALUE;
    for (int r = 0; r < nums.length; r++) {
        sum += nums[r];
        while (sum >= target) {
            minLen = Math.min(minLen, r - l + 1);
            sum -= nums[l++]; // shrink window from left
        }
    }
    return minLen == Integer.MAX_VALUE ? 0 : minLen;
}
// Time: O(n), Space: O(1)
// Each element is added once and removed once -> O(n) despite nested while

```

Longest Substring with At Most K Distinct Characters

```

public int lengthOfLongestSubstringKDistinct(String s, int k) {
    Map<Character, Integer> freq = new HashMap<>();
    int l = 0, maxLen = 0;
    for (int r = 0; r < s.length(); r++) {
        char c = s.charAt(r);
        freq.merge(c, 1, Integer::sum);
        while (freq.size() > k) {
            char leftChar = s.charAt(l++);
            freq.merge(leftChar, -1, Integer::sum);
            if (freq.get(leftChar) == 0) freq.remove(leftChar);
        }
        maxLen = Math.max(maxLen, r - l + 1);
    }
    return maxLen;
}

// Time: O(n), Space: O(k)

```

Key interview tip: always state your brute-force approach first ($O(n^2)$ or $O(n^3)$), then explain how sliding window improves it to $O(n)$. Interviewers want to see your thought process.

End of Part 9 — Coding Questions

PART 10: RESUME DEEP DIVE

This section contains 100 likely interview questions based on your resume — covering Accenture, Deepta AI, Java/Spring Boot, Go Backend, FastAPI, PostgreSQL, Redis, Kafka, ERP systems, and AI/RAG projects. For each question you'll find: a model answer, follow-up questions, red flags interviewers may spot, and how to handle partial knowledge confidently.

How to Handle Partial Experience Confidently

For topics you have partial or theoretical knowledge: say 'I've worked with this at a conceptual level / used it through a framework layer, but haven't configured it from scratch. Here's what I understand about it, and I'm actively deepening this.' This is far better than bluffing or going silent.

- Never say 'I don't know' and stop — always say what you DO know and show curiosity.
- Anchor every answer in something real: 'In my Accenture project' or 'When I built the RAG pipeline at Deepta AI'.
- If asked about something you only know theoretically, say so explicitly — Infosys interviewers respect honesty paired with a solid conceptual understanding.

Section A: Accenture Experience (Java Spring Boot, ERP, PostgreSQL)

MUST KNOW

Q1. Tell me about your role at Accenture. What did you build?

Model Answer: "At Accenture, I worked as a Software Engineer in the [your team] domain, building backend services using Java 11 and Spring Boot. My primary work involved [describe your actual project — e.g., building REST APIs for an ERP module, migrating legacy data pipelines, integrating with third-party services]. I worked closely with [team size] developers in an Agile environment with [2-week sprints / Scrum]. My contributions included [X, Y, Z specific features or modules]."

Red Flag: Red flags: Vague answers like 'I worked on Java stuff'. Be specific about what the service did, what business problem it solved, and your personal contribution.

Q2. What was the most complex technical challenge you solved at Accenture?

Model Answer: Pick ONE real complex problem. Structure it as STAR (Situation, Task, Action, Result). Example: "We had a performance issue where a batch job processing 100,000 records took 4+ hours. I analyzed it and found it was doing N+1 queries — one DB call per record. I rewrote it to use a single JOIN FETCH query with batching, and the same job completed in 25 minutes. I also added an index on the join column which further reduced DB load by 60%."

Red Flag: Don't give a vague challenge. Interviewers want specific numbers, tools, and your personal decision-making.

Q3. How did you design the REST APIs in your Accenture project?

Model Answer: "I followed RESTful conventions — resources mapped to plural nouns (/api/v1/employees), HTTP verbs used semantically (GET to read, POST to create with 201 response, PUT for full updates, PATCH for partial, DELETE with 204). I used DTOs to decouple the API contract from the internal entity model. Request validation was done using @Valid and Bean Validation annotations. Error responses followed a consistent structure via @ControllerAdvice."

Red Flag: Watch out for: returning HTTP 200 for everything, exposing JPA entities directly in API response, not knowing the difference between PUT and PATCH.

Q4. How did you handle database interactions — raw SQL or ORM?

Model Answer: "We used Spring Data JPA with Hibernate as the ORM for most CRUD operations. For complex reporting queries and bulk operations where JPA generated inefficient SQL, I wrote native @Query with PostgreSQL-specific optimizations. I was careful about LAZY loading to avoid N+1 problems — using JOIN FETCH in custom queries when I needed related data. For analytics/reporting, we sometimes bypassed JPA entirely and used JdbcTemplate with raw SQL for performance."

Red Flag: Not knowing what ORM stands for or what Hibernate does. Not being able to explain why you'd use native SQL over JPQL.

Q5. Describe how you used PostgreSQL in your project. What features did you use?

Model Answer: "PostgreSQL was our primary relational database. Beyond standard CRUD, I used PostgreSQL-specific features including: JSONB columns for semi-structured data (flexible metadata per record), window functions like RANK() OVER (PARTITION BY) for reporting queries, CTEs for complex multi-step queries, EXPLAIN ANALYZE to profile and tune slow queries, and partial indexes on frequently filtered status columns. We had a connection pooling setup using HikariCP configured in Spring Boot."

Red Flag: Saying 'I just used it like MySQL' — PostgreSQL has specific features worth mentioning. Not knowing what connection pooling is.

Q6. How did you manage database migrations in your Spring Boot projects?

Model Answer: "We used Flyway for database migration management. Each schema change was written as a versioned SQL migration file (V1__create_employees_table.sql, V2__add_salary_column.sql). Flyway runs on application startup and applies any pending migrations in order. This gave us versioned, reproducible schema changes that could be applied consistently across dev, staging, and production environments, and tracked in source control alongside the code."

Red Flag: Not knowing what Flyway or Liquibase is — interviewers may probe for this since it's standard in Spring Boot projects.

Q7. How did you write unit tests for your Spring Boot services?

Model Answer: "I wrote unit tests using JUnit 5 and Mockito. For service layer tests, I used `@ExtendWith(MockitoExtension.class)` with `@Mock` for dependencies and `@InjectMocks` for the service under test. I avoided spinning up the full Spring context for unit tests — that's `@SpringBootTest` territory and is slower. For integration tests of REST controllers, I used `MockMvc` with `@WebMvcTest`. Test coverage was enforced as part of our CI pipeline via JaCoCo."

Red Flag: Not knowing the difference between unit tests and integration tests. Not being able to explain `@Mock` vs `@MockBean`.

Q8. What was your Agile/Scrum experience at Accenture?

Model Answer: "We followed Scrum with 2-week sprints. I participated in daily standups, sprint planning (estimating stories in story points), sprint review demos, and retrospectives. I maintained my own Jira board, tracked blockers, and coordinated with QA for testing in the same sprint. I was involved in writing user stories for features I owned and breaking them into technical tasks."

Red Flag: Saying 'we just had meetings' without being able to describe the ceremonies or how you personally contributed to the process.

Q9. Have you worked with ERP systems? What was your role?

Model Answer: "Yes, at Accenture I worked on [describe the ERP context — e.g., SAP-adjacent integrations, custom ERP modules, data migration from legacy ERP]. My role specifically involved [your specific contribution — e.g., building APIs to expose ERP data to external systems, transforming data between ERP schema and our domain model, writing batch jobs to sync data from ERP to our reporting database]. The key challenge was [specific challenge: complex data mappings, handling ERP's batch-centric architecture with REST expectations, data consistency]."

Red Flag: Overstating ERP expertise if you only integrated with it rather than building core ERP functionality. Be precise about what layer you touched.

Q10. How did you handle security in your Accenture APIs?

Model Answer: "API security was handled via JWT authentication — clients authenticated once and received a JWT token, which was validated on each subsequent request in a custom Spring Security filter. Role-based access control was implemented using Spring Security's `@PreAuthorize` annotations with roles like `ROLE_ADMIN`, `ROLE_USER`. Sensitive data was never logged. For internal service-to-service calls, we used mutual TLS or API keys managed via a secrets manager. We also had request logging with correlation IDs for audit trails."

Red Flag: Not knowing how JWT works at a basic level. Saying 'Spring Security handles everything' without being able to describe what you configured.

Section B: Deepta AI — AI/RAG Projects

MUST KNOW

Q11. What is a RAG system, and why did you build one?

Model Answer: "RAG — Retrieval-Augmented Generation — is an architecture that enhances Large Language Model responses by first retrieving relevant context from a knowledge base and including it in the prompt. The problem it solves: LLMs have a training cutoff and can hallucinate facts. With RAG, when a user asks a question, we first search our document store for relevant chunks, pass them as context to the LLM, and the model answers based on that grounded context rather than just its training data. At Deepta AI, we built this to [describe your specific use case — e.g., allow users to query internal company documents, build a customer support bot grounded in product documentation]."

Red Flag: Not being able to explain WHY RAG is needed vs just prompting an LLM directly. Be clear about the hallucination problem RAG addresses.

Q12. Walk me through the complete RAG pipeline you built.

Model Answer: "Our RAG pipeline had these stages: (1) Document Ingestion: PDFs and text files were uploaded and split into overlapping chunks (e.g., 512 tokens with 50-token overlap) using a text splitter. (2) Embedding Generation: each chunk was sent to an embedding model (we used [OpenAI text-embedding-ada-002 / sentence-transformers / etc.]) to get a dense vector representation. (3) Vector Storage: embeddings stored in [Pinecone / Weaviate / pgvector in PostgreSQL] with metadata (source document, page number). (4) Query Processing: user query is embedded using

the same model. (5) Retrieval: cosine similarity search finds top-K most relevant chunks from the vector store. (6) Augmented Generation: retrieved chunks + user query assembled into a prompt, sent to GPT-4 / Claude. (7) Response returned to user, often with source citations."

Red Flag: Not knowing what an embedding is. Not knowing what cosine similarity means. Not being able to explain chunk overlap and why it matters.

Q13. What is a vector database, and which one did you use?

Model Answer: "A vector database is specialized for storing and querying high-dimensional vector embeddings via similarity search (finding the nearest vectors to a query vector). Unlike relational databases which search by exact value, vector databases use algorithms like HNSW (Hierarchical Navigable Small World) or IVF to perform approximate nearest-neighbor search efficiently even with millions of vectors. We used [Pinecone / Weaviate / Chroma / pgvector]. I chose [your choice] because [reason — e.g., pgvector because we already had PostgreSQL and the scale didn't require a dedicated service; Pinecone because we needed managed, serverless scaling without ops overhead]."

Red Flag: Not knowing what ANN (Approximate Nearest Neighbor) is or why exact search is impractical for large vector spaces.

Q14. How did you handle chunk size and overlap in your RAG system, and why does it matter?

Model Answer: "Chunk size is critical for retrieval quality. Too small: each chunk lacks enough context for the LLM to answer well. Too large: irrelevant content dilutes the relevant part, and you hit LLM token limits faster. We experimented with 256-512 token chunks. Overlap (e.g., 50 tokens) ensures that sentences or concepts split across chunk boundaries aren't lost — the same sentence appears in both neighboring chunks, maintaining semantic continuity. The optimal chunk size depends on the document type — technical docs with dense information often need larger chunks; FAQs work well with smaller ones. We validated chunk quality by checking retrieval precision on a test query set."

Red Flag: Saying 'we just used default settings' without knowing what those settings are or why they matter.

Q15. How did you build the backend for your AI/RAG project?

Model Answer: "The backend was built with [FastAPI / Spring Boot — mention yours]. I built REST APIs for: document upload and ingestion (/api/documents/upload), query endpoint (/api/query that triggers the full RAG pipeline), and conversation history

management for multi-turn chats. The pipeline was orchestrated using [LangChain / LlamaIndex / custom code]. For async heavy operations like large file ingestion (which could take seconds to minutes), I used background tasks [FastAPI BackgroundTasks / Spring @Async] so the API returned immediately with a job ID and the user polled for status."

Red Flag: Not knowing how LangChain or a similar orchestration framework works at a conceptual level.

Q16. What was your experience with FastAPI? How does it compare to Spring Boot?

Model Answer: "I used FastAPI for Python-based microservices, primarily for AI/ML-heavy workloads where the Python ecosystem (LangChain, HuggingFace, numpy) made more sense than Java. FastAPI's key strengths: automatic OpenAPI/Swagger documentation from type hints, Pydantic models for validation (similar to Spring's @Valid + Bean Validation), async/await native support with Python's asyncio (great for I/O-heavy LLM API calls), and very fast startup with minimal boilerplate. Compared to Spring Boot: FastAPI is lighter and more pythonic; Spring Boot has a richer ecosystem for enterprise Java, better ORM support (Hibernate/JPA), and Spring Security for complex auth. I choose based on the team and workload — Java/Spring for enterprise backend, Python/FastAPI for AI/ML services."

Red Flag: Not knowing what Pydantic is. Saying FastAPI is just 'Python Flask' — FastAPI is fundamentally different with async support and type hints.

Q17. How did you prevent hallucinations in your RAG system?

Model Answer: "Hallucination prevention was a key concern. Our approaches: (1) Retrieval grounding — the LLM prompt explicitly instructed it to answer ONLY from the provided context and say 'I don't know' if context didn't contain the answer. (2) Source citations — every response included which document chunk it came from, making it verifiable. (3) Confidence threshold — if the similarity score of the top retrieved chunk was below a threshold, we showed a 'low confidence' warning. (4) Re-ranking — we used a cross-encoder re-ranker to more accurately score retrieved chunks before passing to the LLM, improving relevance. (5) Evaluated with test queries and tracked hallucination rate before and after each pipeline change."

Red Flag: Saying 'RAG automatically prevents hallucinations' — it reduces but doesn't eliminate them. Show you understand the nuance.

Q18. How did you evaluate the quality of your RAG system?

Model Answer: "RAG evaluation had multiple dimensions: (1) Retrieval quality — did we retrieve the right chunks for a given query? Measured by recall@K (are the relevant

chunks in the top K results?). (2) Answer faithfulness — does the generated answer stay grounded in the retrieved context (not hallucinate)? (3) Answer relevance — does the answer actually address the question? We used [RAGAS framework / custom evaluation scripts with GPT-4 as judge] to automate evaluation against a curated test set of question-answer pairs with known ground truth. We tracked these metrics in each iteration and used them to tune chunk size, overlap, and model parameters."

Red Flag: Not having a systematic evaluation approach — 'we just tested it manually' is a red flag for a production AI system.

Q19. Describe your Go backend experience. What did you build and why Go?

Model Answer: "I built [describe your Go service — e.g., a high-throughput event processing service, a lightweight API proxy, a file processing service] in Go. I chose Go for this service because: (1) exceptional performance for I/O-bound concurrent work — goroutines are extremely lightweight (2KB vs ~1MB for Java threads), and Go's runtime schedules thousands of them efficiently; (2) fast binary compilation and startup — critical for this service's deployment model; (3) minimal dependencies and small Docker image size. I implemented [specific features] using Go's standard library and [specific packages like gin/echo for HTTP, database/sql for DB access, encoding/json for serialization]."

Red Flag: Not knowing what makes Go different from Java — the goroutine model, static typing without JVM overhead, and compilation to native binary are key differentiators to mention.

Q20. How do goroutines in Go compare to Java threads?

Model Answer: "Goroutines are much lighter than Java threads — roughly 2KB initial stack vs ~256KB-1MB for a Java thread. Go's runtime manages goroutine scheduling using M:N threading (multiplexing many goroutines onto fewer OS threads via a scheduler), meaning you can have hundreds of thousands of goroutines without exhausting OS resources. In Java, even with virtual threads (Java 21 Project Loom), you're still configuring thread pools and worrying about blocking. Go's channels make inter-goroutine communication safe and idiomatic — instead of shared mutable state with locks, you 'share memory by communicating'. The Go runtime handles goroutine scheduling transparently, whereas in Java you explicitly manage thread pools via `ExecutorService`."

Red Flag: Not knowing the basic goroutine model or confusing goroutines with threads.

Section C: Java Spring Boot Deep Dive

MUST KNOW

Q21. Why would you use Spring Boot over plain Spring Framework?

Model Answer: "Spring Boot is opinionated Spring — it auto-configures common components based on what's on the classpath, provides starter POMs (spring-boot-starter-web pulls in embedded Tomcat, Jackson, Spring MVC, etc.), and gives you a runnable standalone JAR with an embedded server — no WAR deployment needed. You spend time on business logic instead of XML bean definitions and server setup. Features like Actuator (health checks, metrics), DevTools (hot reload), and production-ready defaults are bundled. Plain Spring gives more control but requires significantly more manual setup."

Red Flag: Saying 'Spring Boot is faster' without explaining what makes it faster (auto-configuration, embedded server, starters).

Q22. What is @SpringBootApplication and what does it do?

Model Answer: "@SpringBootApplication is a convenience annotation combining three annotations: @Configuration (marks this class as a source of Spring bean definitions), @EnableAutoConfiguration (enables Spring Boot's auto-configuration mechanism — scans classpath, configures beans automatically), and @ComponentScan (scans the package and sub-packages for @Component, @Service, @Repository, @Controller annotated classes to register as beans). So when you run the main class, Spring Boot bootstraps the ApplicationContext, auto-configures everything, and starts the embedded server."

Red Flag: Not knowing what the three underlying annotations are — interviewers probe this to see if you understand what's happening under the hood.

Q23. What is Spring AOP and where have you used it?

Model Answer: "AOP — Aspect-Oriented Programming — lets you extract cross-cutting concerns (logging, security checks, transaction management, performance monitoring) into separate Aspects, keeping them out of your business logic. Key concepts: Aspect (class with cross-cutting logic), Advice (the code to run — @Before, @After, @Around, @AfterThrowing), Pointcut (expression defining which method executions to intercept). Spring AOP uses dynamic proxy-based weaving at runtime. I've used it for: (1) execution time logging on all @Service methods via @Around advice, (2) audit logging — log who called what method with what parameters, (3) @Transactional itself is implemented via AOP in Spring."

Red Flag: Not knowing that @Transactional works via AOP — this connects concepts and shows depth.

Q24. How would you implement pagination in a Spring Boot API?

Model Answer: "Spring Data JPA has first-class pagination support. The repository method accepts a Pageable parameter and returns a Page<T>. The controller accepts page number, size, and sort params via @RequestParam or directly via Pageable resolution."

Red Flag: Returning an entire list and paginating in Java application code — that pulls all data from DB, defeating the purpose. Pagination must happen at DB query level (LIMIT + OFFSET).

Q25. What is the difference between @Component, @Service, @Repository, and @Controller?

Model Answer: "All four are @Component stereotypes — Spring scans and registers them all as beans. The semantic difference: @Component is generic; @Service marks business logic layer (no extra behavior, just semantics for developer clarity); @Repository marks DAO/persistence layer and ALSO enables Spring's persistence exception translation — converts DB-specific exceptions (SQLException) to Spring's DataAccessException hierarchy; @Controller marks an MVC controller and is processed by DispatcherServlet; @RestController = @Controller + @ResponseBody for REST APIs."

Red Flag: Saying they're all the same — @Repository has the important extra behavior of exception translation.

Q26. What is Spring Boot Actuator? Have you used it?

Model Answer: "Spring Boot Actuator exposes production-ready management endpoints for monitoring and operating the application. Key endpoints: /actuator/health (overall health including DB, disk, cache — used by load balancers and Kubernetes liveness/readiness probes), /actuator/metrics (counters, gauges, timers — integrated with Prometheus/Grafana), /actuator/info (app version, build info), /actuator/env (environment properties), /actuator/loggers (change log levels at runtime without restart). In my project, I used /health and /metrics endpoints, exposed them to our monitoring stack, and secured the actuator endpoints (especially /env) to internal network access only."

Red Flag: Not knowing Actuator at all — it's standard in production Spring Boot apps.

Q27. How do you handle application configuration and secrets in Spring Boot?

Model Answer: "Application config is managed via application.properties / application.yml, with profile-specific files (application-dev.yml, application-prod.yml) for

environment-specific settings. `@Value` injects individual properties; `@ConfigurationProperties` binds a group of related properties to a class with type safety. For secrets (DB passwords, API keys), we NEVER put them in committed config files. In production: environment variables passed at startup (Spring reads `SPRING_DATASOURCE_PASSWORD` from env), or a secrets manager like AWS Secrets Manager / HashiCorp Vault, with Spring Cloud Vault or AWS SDK integration to load secrets at startup."

Red Flag: Saying 'we put passwords in application.properties and checked them into Git' — security red flag.

Q28. What is the difference between `@RestTemplate` and `WebClient` in Spring Boot?

Model Answer: "RestTemplate is Spring's traditional blocking HTTP client — it makes an HTTP call and the thread waits (blocked) until the response returns. It's simpler to use but doesn't scale as well under high concurrency because each call blocks a thread. WebClient is Spring WebFlux's non-blocking, reactive HTTP client — it returns a `Mono/Flux` and the thread is freed while waiting for the response, making it far more efficient for high-concurrency scenarios or when calling multiple services in parallel. As of Spring 5, RestTemplate is in maintenance mode and WebClient is the recommended alternative, even in non-reactive apps, because you can use `.block()` to get a blocking result while still benefiting from connection pooling and better resource management."

Red Flag: Not knowing the blocking vs non-blocking distinction. Not knowing RestTemplate is deprecated in favor of WebClient.

Q29. What is Spring Cloud Config, and have you used it?

Model Answer: "Spring Cloud Config provides centralized external configuration for distributed applications. Instead of each microservice having its own application.yml, all configs live in a Git repository (or Vault, S3). The Config Server reads from there and serves configs to all services via HTTP. Services use Spring Cloud Config Client to fetch their config on startup. Benefits: single source of truth for config, versioned config changes via Git history, ability to refresh config without restarting services using `@RefreshScope` + Spring Cloud Bus (posts a refresh event to all instances via Kafka/RabbitMQ). I've [used it / been exposed to it at Accenture / implemented a lighter version using environment variables — be honest about your actual level of use]."

Red Flag: Claiming deep Config Server experience if you only used it at the surface — be honest about the scope of your use.

Q30. How did you implement logging in your Spring Boot application?

Model Answer: "We used SLF4J as the logging facade with Logback as the implementation (the default in Spring Boot). I used `@Slf4j` (Lombok annotation) to inject a logger without boilerplate. Key practices: (1) Structured logging with MDC (Mapped Diagnostic Context) to include request correlation IDs in every log line — when tracing a request across logs, correlation ID links all related entries. (2) Log levels used appropriately: DEBUG for detailed diagnostics, INFO for normal operations, WARN for unexpected but recoverable situations, ERROR for failures requiring attention — never just `System.out.println` in production. (3) Logs shipped to a central platform (ELK stack / Splunk / CloudWatch) for search and alerting. (4) Never log sensitive data like passwords, PII, or tokens."

Red Flag: Using `System.out.println` in production code — interviewers see this as a junior mistake. Not knowing what MDC/correlation ID is.

Section D: PostgreSQL and Redis Deep Dive

MUST KNOW

Q31. How did you tune PostgreSQL query performance in your project?

Model Answer: "My performance tuning process: (1) Identify slow queries using `pg_stat_statements` or application monitoring — find queries taking `>100ms`. (2) Run `EXPLAIN ANALYZE` to see the actual query plan — look for Seq Scan on large tables (missing index), high Rows Removed (bad selectivity), hash joins vs nested loops (different for different data sizes). (3) Add appropriate indexes — btree for equality/range, GIN for JSONB/full-text, partial index if querying a subset. (4) Check for N+1 patterns in JPA and fix with `JOIN FETCH`. (5) For reporting queries: use read replicas to offload analytics traffic. (6) Use `VACUUM ANALYZE` on tables with high churn to update planner statistics."

Red Flag: Not knowing what `EXPLAIN ANALYZE` does. Not knowing what a sequential scan is or why it's slow on large tables.

Q32. What is HikariCP and why does it matter in Spring Boot?

Model Answer: "HikariCP is a high-performance JDBC connection pool — the default in Spring Boot since 2.x. Creating a database connection is expensive (TCP handshake, authentication, SSL). HikariCP maintains a pool of pre-created connections that are reused by application threads. Key config: `maximumPoolSize` (max connections — typically 10-20 for most apps), `minimumIdle` (connections kept ready even at low load), `connectionTimeout` (how long to wait for a connection from the pool before throwing an exception), `idleTimeout`, `maxLifetime`. Tuning HikariCP is critical for high-throughput Spring Boot services — too small a pool and threads wait; too large and you exhaust PostgreSQL's `max_connections`."

Red Flag: Not knowing what connection pooling is or why it's needed — basic production knowledge.

Q33. How did you use Redis in your project? What data structure did you use?

Model Answer: "In my project, I used Redis for [be specific]: (1) Response caching with Spring's `@Cacheable` — expensive DB queries or third-party API results cached in Redis with TTL, reducing latency from ~200ms to ~2ms on cache hits. (2) Rate limiting — `INCR + EXPIRE` on a per-user key to enforce API call limits. I used Redis String type for caching (serialized JSON), and potentially Hash for partial cache invalidation (store different fields separately). Configured Spring Data Redis with Lettuce client (reactive, connection pooling), and set appropriate TTLs per cache to prevent stale data buildup."

Red Flag: Not knowing which Redis data structure you used or why (String vs Hash vs List, etc.).

Q34. What is cache eviction policy in Redis and which did you configure?

Model Answer: "When Redis reaches its maxmemory limit, it needs to evict keys to make room. Common policies: `noeviction` (return error on new writes — bad for cache), `allkeys-lru` (evict least recently used key from all keys — good general cache behavior), `volatile-lru` (evict LRU from only keys with TTL set), `allkeys-lfu` (least frequently used — better for Zipfian distributions where some keys are accessed much more than others), `volatile-ttl` (evict keys with shortest TTL first). In my project I used [your actual config, or: `allkeys-lru` for the response cache since we wanted the least recently accessed items evicted, which is intuitive for a cache]. Setting maxmemory and the eviction policy explicitly is important — otherwise Redis can fill up and start returning errors."

Red Flag: Not knowing eviction policies exist or setting up Redis without a maxmemory limit in production.

Q35. How do you prevent cache stampede (thundering herd) in Redis?

Model Answer: "Cache stampede happens when a popular cached item expires and thousands of requests simultaneously hit the DB to refill it. Solutions: (1) Probabilistic early expiry — with some probability, refresh the cache slightly before it expires, so one request refreshes while others still get cached data. (2) Redis distributed lock (`SET NX EX`) — only one request gets the lock to populate the cache, others wait briefly and then get the freshly cached value. (3) Cache pre-warming — populate the cache proactively before expiry (schedule a job). (4) Staggered TTLs — add random jitter to TTL (e.g., `base TTL + random 0-60 seconds`) so all items don't expire at the same instant, especially important after a cache flush."

Red Flag: Not knowing what a cache stampede is — a production readiness question interviewers ask for senior-ish candidates.

Section E: Kafka Deep Dive

MUST KNOW

Q36. Why did you choose Kafka over RabbitMQ or a simple database queue for your use case?

Model Answer: "I chose Kafka because [be specific to your project — here's a template]: we needed high-throughput event streaming where multiple independent consumers needed to read the same events (e.g., Order Service events needed by both Payment Service and Notification Service independently — Kafka's consumer groups let each read independently). We also needed message replay capability — if the notification service went down, Kafka's retention (e.g., 7 days) let it pick up where it left off on restart, which wouldn't be possible with a standard queue that deletes on consume. The partitioned log model also let us guarantee order per entity (using entity ID as partition key). RabbitMQ is better for complex routing and smaller message volumes where you need sophisticated exchange/queue topologies."

Red Flag: Not being able to justify the choice — saying 'we just used Kafka' without knowing why.

Q37. How did you ensure no message loss in your Kafka producer?

Model Answer: "For critical messages, I configured: (1) `acks=all` — wait for all in-sync replicas to acknowledge before returning success, ensuring durability even if one broker fails. (2) `retries=3` (or more) with `retry.backoff.ms` — automatically retry transient failures. (3) `enable.idempotence=true` — ensures that even with retries, no duplicate messages are produced to the same partition (Kafka assigns a producer epoch and sequence number to detect duplicates). (4) On the application side, I wrapped the `send()` in a try-catch and implemented a dead letter queue or fallback storage for sends that failed after all retries. (5) Used transactional producers where exactly-once semantics were required."

Red Flag: Not knowing what `acks=all` means or why idempotent producer matters when retrying.

Q38. How did you handle duplicate message processing in Kafka consumers?

Model Answer: "With at-least-once delivery (the default safe mode), duplicates can occur if a consumer processes a message but crashes before committing the offset. To handle this: (1) Idempotent consumers — design processing so that processing the

same message twice produces the same result (e.g., use UPSERT instead of INSERT, check if event has already been processed using a processed_events table keyed by message ID). (2) Deduplication table — store processed Kafka message keys/IDs in DB; on receipt, check if already processed and skip if so. (3) Exactly-once semantics with Kafka Transactions — for Kafka-to-Kafka processing. (4) Idempotency keys — for outgoing HTTP calls triggered by Kafka messages, use idempotency keys so repeated calls don't create duplicate resources."

Red Flag: Saying 'we used exactly-once, so no duplicates' without knowing the implementation complexity and cost.

Q39. How do you monitor Kafka consumer lag in production?

Model Answer: "Consumer lag is the difference between the latest produced offset and the consumer group's committed offset for each partition — it tells you how far behind consumers are. Monitoring: (1) kafka-consumer-groups.sh CLI tool for manual inspection. (2) Prometheus JMX Exporter on Kafka brokers + Grafana dashboards for real-time lag visualization. (3) Burrow (LinkedIn's consumer lag monitoring tool) for automated alerting on growing lag. (4) In Spring Boot, Kafka Actuator metrics (spring.kafka.listener.*) expose consumer lag through Micrometer. Alerting on lag: alert when lag exceeds threshold for a sustained period (e.g., > 1000 messages lagging for > 5 minutes), which indicates consumer is not keeping up with production rate and needs scaling (add more consumer instances up to partition count)."

Red Flag: Not knowing what consumer lag is or how to observe it — basic operational knowledge for Kafka users.

Q40. If a Kafka consumer is processing slowly and lag is growing, what do you do?

Model Answer: "Diagnose first, then act: (1) Profile the consumer code — is processing itself slow (CPU-bound computation, slow external API call, slow DB write)? Fix the processing logic or make it async. (2) Check if it's I/O-bound — the consumer waits on DB/HTTP calls. Solution: increase concurrency in @KafkaListener (concurrency=N to have N threads each processing from assigned partitions) — but max N = number of partitions. (3) Scale out — add more consumer instances (pods/containers) to the same consumer group; Kafka rebalances partitions across them. (4) Increase partitions if current count limits parallelism (note: can only increase, not decrease). (5) If processing is fundamentally slow, decouple: consumer receives message quickly, writes to a queue or DB table, and worker threads process asynchronously from that internal queue."

Red Flag: Saying 'just increase partitions' as the first answer — you should diagnose before scaling.

Section F: System Design & Situational Questions

MUST KNOW

Q41. Design a URL shortening service (like bit.ly). Walk me through the key components.

Model Answer: "Core requirements: create short code for long URL, redirect short URL to original, analytics (optional). (1) API: POST /shorten (body: longUrl) returns shortUrl; GET /{code} returns redirect (301/302 to longUrl). (2) ID generation: encode a unique ID (from DB auto-increment or distributed ID generator like Snowflake) to a base62 string (a-z, A-Z, 0-9) — 7 characters gives $62^7 = \sim 3.5$ trillion codes. (3) DB: simple table: code, longUrl, createdAt, userId. (4) Cache: Redis with code->longUrl mapping; TTL per entry. On redirect: check Redis first, fallback to DB on miss, populate cache. (5) Redirect type: 302 (temporary) if analytics tracking needed on each click; 301 (permanent) for browser caching. (6) Scale: stateless API pods behind load balancer, Redis for read scaling, DB read replicas."

Red Flag: Not thinking about the encoding approach or cache strategy — these are the key design decisions.

Q42. How would you design a notification service that sends email, SMS, and push notifications?

Model Answer: "Key principle: decouple notification triggering from sending using a message queue (Kafka). (1) Any service publishes a notification event to Kafka: {type: 'ORDER_CONFIRMED', userId: 123, orderId: 456}. (2) NotificationService consumes events from Kafka. (3) User Preferences lookup: query user's notification preferences (DB) — does user want email? SMS? Push? Respect opt-outs. (4) Template rendering: load notification template, populate with event data. (5) Channel dispatchers: EmailDispatcher (via SES/SendGrid), SMSDispatcher (via Twilio), PushDispatcher (via FCM/APNs). (6) Retry logic with dead letter topic for failed sends. (7) Delivery status tracking in DB. Benefits: Kafka decouples senders from notification service (any service can trigger notifications), each channel scales independently, event replay if notification service goes down."

Red Flag: Designing synchronous notification sending from within business logic — coupling and slowness.

Q43. How would you implement an API rate limiter?

Model Answer: "Using Redis Token Bucket or sliding window counter: For sliding window — key = 'rate_limit:{userId}:{currentMinute}'. INCR key, set EXPIRE on first increment. If count > limit, return 429. Include Retry-After header indicating when limit resets. For token bucket: store last refill time and tokens remaining in Redis Hash; on each request, calculate tokens replenished since last refill, subtract 1 for this request,

update atomically using Redis Lua script (to avoid race conditions between read-modify-write). At API Gateway level: Spring Cloud Gateway has built-in Redis-backed rate limiter (RequestRateLimiter filter with RedisRateLimiter) — easier than manual implementation."

Red Flag: Using a local in-memory map for rate limiting in a multi-instance deployment — limits don't aggregate across instances.

Q44. How would you implement distributed tracing across microservices?

Model Answer: "Distributed tracing tracks a request as it flows through multiple microservices. Key concepts: Trace ID (unique per request journey), Span ID (per service/operation within the trace), Parent Span ID (linking child to parent). Implementation with Spring Boot: (1) Add Spring Cloud Sleuth (or Micrometer Tracing in Spring Boot 3) dependency — auto-instruments RestTemplate, WebClient, Kafka messages to propagate trace/span IDs via HTTP headers (B3 format: X-B3-TraceId, X-B3-SpanId). (2) Configure exporter to send spans to Zipkin or Jaeger. (3) Each service receives trace headers from incoming requests, creates a child span, and propagates headers to downstream calls. (4) Jaeger/Zipkin UI shows the full request tree across all services with timing info. In production, typically sample only 1-10% of traces to control volume."

Red Flag: Not knowing what a trace ID or span is. Not knowing how distributed tracing differs from logging.

Q45. How would you handle a production incident where your Spring Boot service is returning 500 errors?

Model Answer: "Incident response steps: (1) Assess impact: how many users affected? Which endpoints? When did it start? (2) Check logs immediately — centralized logging (ELK/CloudWatch); filter by ERROR level; look for the exception and stack trace. (3) Check metrics — is CPU/memory spiking? DB connection pool exhausted? Response time spiking? (4) Check recent deployments — was this triggered by a recent code push or config change? If yes, consider rollback. (5) Check downstream dependencies — is a DB, external API, or Kafka consumer the root cause? (6) Apply immediate mitigation if possible (restart instance, scale out, toggle feature flag off). (7) Root cause analysis after stabilization — write a post-mortem with timeline, root cause, impact, and action items to prevent recurrence."

Red Flag: Starting with 'restart the server' without understanding the cause — shows no diagnostic discipline.

Section G: Additional 55 High-Value Questions (Q46–Q100)

The following questions are presented in a condensed format — study each question, then formulate your answer anchoring it in your real project experience.

MUST KNOW

Q46. [Core Java] What's the difference between HashMap and ConcurrentHashMap?

HashMap: not thread-safe, single-threaded use. **ConcurrentHashMap:** thread-safe, achieved by segment-level locking (Java 7) or CAS + synchronized per-bin (Java 8) — much better concurrency than Hashtable (which locks the entire map). **ConcurrentHashMap** allows concurrent reads without any lock, and concurrent writes to different buckets. Null keys and values NOT allowed (unlike HashMap). Use **ConcurrentHashMap** when multiple threads read/write a shared map.

Q47. [Core Java] What is Optional in Java 8, and why was it introduced?

Optional<T> is a container that either contains a value or is empty. Introduced to explicitly represent 'might be absent' instead of returning null (which causes NPE when callers forget to check). Key methods: **Optional.of(val)**, **Optional.ofNullable(val)**, **isPresent()**, **get()**, **orElse(default)**, **orElseGet(supplier)**, **orElseThrow()**, **map()**, **filter()**. Best practice: use as return type from methods that might not find a result (e.g., **repository.findById**) — NOT as parameter type or field type.

Q48. [Core Java] Explain Java 8 Stream API with an example from your project.

Stream API enables functional-style operations on collections. Key ops: **filter()** (predicate), **map()** (transform), **flatMap()** (flatten), **collect()** (terminal — to list/map/set), **reduce()** (fold), **sorted()**, **distinct()**, **limit()**, **anyMatch/allMatch/noneMatch()**. Example: filter active employees, group by dept, compute avg salary per dept — all in one readable pipeline with **Collectors.groupingBy** + **Collectors.averagingDouble**. Streams are lazy — intermediate ops evaluated only when terminal op is called.

Q49. [Core Java] What are lambda expressions and functional interfaces?

A lambda is a concise anonymous function: (params) -> body. Functional interfaces have exactly one abstract method (e.g., **Predicate<T>**, **Function<T,R>**, **Supplier<T>**, **Consumer<T>**, **Comparator<T>**). **@FunctionalInterface** annotation enforces the single-abstract-method contract at compile time. Lambdas enable functional-style code and are the basis of the Stream API. Method references (**ClassName::method**) are shorthand lambdas.

Q50. [Core Java] What is the difference between Comparable and Comparator?

Comparable: implemented by the class itself (implements `Comparable<T>`), defines natural ordering via `compareTo()`. Only one ordering possible per class. **Comparator:** external, defines custom ordering strategy via `compare()`. Can have multiple Comparators for different sort criteria. Use Comparable when there's one obvious natural ordering (e.g., employee by ID). Use Comparator for multiple sort strategies or when you can't modify the class. Java 8:

`Comparator.comparing(Employee::getSalary).thenComparing(Employee::getName).`

Q51. [Spring Boot] What is Spring Boot auto-configuration and how does it work internally?

When `spring-boot-autoconfigure` is on the classpath, Spring Boot scans `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` file for a list of auto-configuration classes. Each class uses `@ConditionalOn...` annotations (`@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty`) to decide whether to activate. For example, `DataSourceAutoConfiguration` activates only if a JDBC driver class is on the classpath AND no `DataSource` bean is already defined. This is why `spring-boot-starter-*` dependencies trigger the right configs automatically.

Q52. [Spring Boot] How would you implement asynchronous processing in Spring Boot?

Use `@Async` on a method in a `@Service` bean + `@EnableAsync` on a `@Configuration` class. Spring wraps the method in a proxy that submits it to a `ThreadPoolTaskExecutor` (configure your own via `@Bean` for production control). Return `CompletableFuture<T>` from async methods to allow callers to chain callbacks or wait with `.get()`. Key gotcha: `@Async` doesn't work on self-invocation (same class calls same class). Configure thread pool: `corePoolSize`, `maxPoolSize`, `queueCapacity`, `threadNamePrefix` for visibility in monitoring.

Q53. [Spring Boot] What is Spring Boot DevTools and when do you use it?

DevTools adds developer experience features: automatic restart on classpath changes (faster than full restart, uses two classloaders), LiveReload server to refresh browser, relaxed template caching disabled (so Thymeleaf/Freemarker changes show instantly). Uses conditional `@ConditionalOnLocalHostAddress` so DevTools auto-excludes itself in production (not running from embedded JAR). Don't include in production builds — it's a dev-only dependency (optional: true in Maven or devOnly in Gradle).

Q54. [SQL] What is a stored procedure vs a function in PostgreSQL?

Both are named server-side routines. Function: returns a value, can be used in SELECT/WHERE, cannot call COMMIT/ROLLBACK (no transaction control within function by default). Procedure (PostgreSQL 11+): does not return a value via RETURN, supports transaction control (CALL proc; inside a transaction), called with CALL statement. In Spring Boot with JPA, call procedures via @NamedStoredProcedureQuery or SimpleJdbcCall. In practice, prefer application-side logic over stored procedures for better testability and version control.

Q55. [SQL] What is a CTE (Common Table Expression) and when do you prefer it over a subquery?

A CTE (WITH clause) defines a named temporary result set scoped to one query. WITH dept_avg AS (SELECT dept_id, AVG(salary) as avg FROM employees GROUP BY dept_id) SELECT e.name, da.avg FROM employees e JOIN dept_avg da ON e.dept_id = da.dept_id. Prefer CTE over subquery when: the same subresult is needed multiple times in the query (CTE can be referenced multiple times, subquery must be repeated), the query is complex and CTEs improve readability, or you need recursive queries (WITH RECURSIVE for hierarchical data like org charts).

Q56. [SQL] Write a query to find all managers who manage more than 3 employees.

SELECT manager_id, COUNT(*) as report_count FROM employees WHERE manager_id IS NOT NULL GROUP BY manager_id HAVING COUNT(*) > 3; If you need manager names: SELECT m.name, COUNT(*) FROM employees e JOIN employees m ON e.manager_id = m.emp_id WHERE e.manager_id IS NOT NULL GROUP BY m.emp_id, m.name HAVING COUNT(*) > 3;

Q57. [Microservices] What is the difference between orchestration and choreography in microservices?

Choreography: services react to events published by others — no central coordinator. Decentralized, loose coupling, harder to trace the overall flow (must read multiple services' code to understand flow). Orchestration: a central orchestrator directs each step explicitly — easier to visualize and debug the saga flow, but the orchestrator becomes a critical component. Choreography better for simple, stable flows; orchestration better for complex, multi-step business processes where you need clear visibility and error handling.

Q58. [Microservices] How do you handle inter-service communication failures gracefully?

Pattern stack: Retry (with exponential backoff + jitter for transient failures), Timeout (always set timeouts on HTTP calls — never block forever), Circuit Breaker (stop calling a failing service), Fallback (serve degraded response when downstream is down), Bulkhead (isolate resources so one failing service doesn't exhaust thread pool of caller). In Spring Boot: Resilience4j covers all of these. Design for failure: assume any service call can fail and have a plan for each case.

Q59. [Microservices] What is service mesh, and have you used Istio or Linkerd?

A service mesh adds a proxy sidecar (like Envoy) to each service pod. The sidecar handles: mutual TLS between services, load balancing, circuit breaking, distributed tracing, observability — all at the infrastructure level without changing application code. Istio and Linkerd are popular service mesh implementations, typically used with Kubernetes. For 2-3 YOE: be honest if you haven't used it — 'I'm familiar with the concept and know it sits as a sidecar proxy layer. I've worked with application-level resilience via Resilience4j but not Istio directly yet.'

Q60. [Kafka] What is Kafka Streams, and how is it different from a regular Kafka consumer?

Kafka Streams is a library for stateful stream processing — transform, aggregate, join, and window data as it flows through Kafka, all within a Java application. Unlike a regular consumer that just reads and processes messages, Kafka Streams: maintains local state stores (RocksDB-backed), supports windowed aggregations (count events per 5-minute window), joins across topics, and guarantees exactly-once processing with Kafka Transactions. Alternative: Apache Flink (distributed, more complex, for very high scale). For 2-3 YOE: know the concept, mention it as 'stateful stream processing on top of Kafka' with windowed aggregations.

Q61. [Kafka] How do you implement a dead letter topic (DLT) in Spring Kafka?

A DLT receives messages that failed processing after all retries. Spring Kafka's SeekToCurrentErrorHandler (or DefaultErrorHandler in 2.8+) automatically publishes failed messages to a topic named {original-topic}.DLT after N retries. Configure: @Bean DefaultErrorHandler errorHandler() { return new DefaultErrorHandler(new DeadLetterPublishingRecoverer(kafkaTemplate), new FixedBackOff(1000L, 3)); } — retry 3 times with 1s delay, then publish to DLT. Monitor DLT for messages that need manual investigation/replay.

Q62. [Go] What are channels in Go and how do they enable concurrency?

Channels are typed conduits for communication between goroutines — the primary way Go implements the principle 'do not communicate by sharing memory; share memory by communicating.' A goroutine sends values into a channel (`ch <- value`) and another receives (`value := <-ch`). Unbuffered channel: sender blocks until receiver is ready (synchronization). Buffered channel (`make(chan T, capacity)`): sender blocks only when buffer is full. Select statement allows a goroutine to wait on multiple channel operations, choosing whichever is ready first — like a switch for channels.

Q63. [Go] How does error handling in Go differ from Java?

Go doesn't have exceptions. Functions return multiple values, with the last conventionally being an error: `result, err := doSomething(); if err != nil { return fmt.Errorf('context: %w', err) }`. The `%w` verb wraps the error for `errors.Is()` and `errors.As()` unwrapping. Errors are explicit values, not exceptional control flow — callers are forced to handle them. Panics exist (like Java unchecked exceptions) for truly unexpected situations and can be recovered with `recover()` in a deferred function. This explicit error propagation is more verbose but makes the error path visible in the code.

Q64. [Go] What is defer in Go and when do you use it?

`defer` postpones a function call until the surrounding function returns — runs even if there's a panic. Used for: resource cleanup (`defer file.Close()`, `defer rows.Close()`, `defer mutex.Unlock()`), ensuring paired operations, and simplifying cleanup logic (don't need to repeat cleanup in every return path). Deferred calls are executed in LIFO order. A common pattern: `func readFile(path string) error { f, err := os.Open(path); if err != nil { return err }; defer f.Close(); // use f... }` — cleanup is right next to the acquisition, less likely to be forgotten.

Q65. [FastAPI] What is Pydantic and how does it differ from Java's Bean Validation?

Pydantic is Python's data validation library using Python type hints. Define a `BaseModel` subclass with typed fields and Pydantic validates, coerces, and serializes automatically. `class CreateUserRequest(BaseModel): name: str; age: int = Field(ge=18, le=65)`. FastAPI uses Pydantic models for request body parsing and response serialization. Similar to Java's Bean Validation (`@NotNull`, `@Min`) + Jackson for JSON, but tighter integration — type hint IS the validator. Pydantic v2 (much faster, written in Rust) now standard. Key difference: Python's dynamic typing means runtime validation is more critical than Java's compile-time type checking.

Q66. [FastAPI] How does `async/await` work in FastAPI, and when do you use it?

FastAPI runs on Starlette/uvicorn with asyncio. `async def` endpoints are non-blocking — when awaiting I/O (DB query, HTTP call, LLM API call), the event loop can process other requests instead of blocking a thread. Use `async def` for endpoints that do async I/O. Use regular `def` for CPU-bound tasks (FastAPI runs these in a thread pool automatically to avoid blocking the event loop). With `async`: one thread handles thousands of concurrent I/O-bound requests. Key: `await` must only be called inside `async def`. For DB: use `async SQLAlchemy` or `async drivers` (`asyncpg` for PostgreSQL).

Q67. [FastAPI] How do you structure a large FastAPI project? What's equivalent to Spring Boot's layered architecture?

Standard FastAPI project structure mirrors Spring Boot's layered architecture: `routers/` (= controllers — define endpoints with `APIRouter`), `services/` (= service layer — business logic), `repositories/` (= repository layer — DB access), `models/` (= entity + DTO — Pydantic models for request/response, SQLAlchemy models for DB), `core/` (config, security, database session). Dependency injection in FastAPI uses `Depends()` function — similar to Spring's `@Autowired`: `def get_employee(id: int, db: Session = Depends(get_db), service: EmployeeService = Depends(get_employee_service))`. FastAPI's DI is function-parameter based, not annotation-based.

Q68. [ERP] What was your role in the ERP integration? What were the key challenges?

Answer with your specific experience. Template: 'My role was to build/integrate with [specific ERP module — HR, Finance, Inventory]. The key technical challenge was [data transformation complexity between ERP's normalized schema and our domain model / handling ERP's batch-first architecture with our real-time API expectations / mapping ERP business logic to our service layer]. We addressed this by [solution: adapter pattern for data mapping, change data capture via Debezium to sync ERP changes to our DB, etc.].' Focus on what YOU specifically built and decided.

Q69. [System Design] How would you design a system to process 1 million orders per day?

1M orders/day $\approx$ 12 orders/second avg, peaks could be 10x = 120/sec. Approach: (1) API layer: stateless Spring Boot pods behind ALB, auto-scale horizontally; (2) Order creation: `async` — accept request, publish to Kafka, return 202 Accepted + order ID immediately; (3) Kafka: order-events topic with 12+ partitions; (4) Order processing consumers: downstream services (payment, inventory, fulfillment) consume in parallel from their own consumer groups; (5) DB: PostgreSQL with read replicas, connection pooling via HikariCP, shard by `customer_id` if needed; (6) Cache: Redis for product catalog, pricing, inventory counts; (7) Observability: distributed tracing (Jaeger), metrics (Prometheus), alerting on processing lag.

Q70. [System Design] What's the difference between horizontal and vertical scaling, and which did you use in your projects?

Vertical: add more CPU/RAM to existing server. Simple, no code changes, limited by hardware ceiling, single point of failure. **Horizontal:** add more instances/nodes. Requires stateless services, external session storage, load balancing — but theoretically unlimited scale and better fault tolerance. In my projects: the Spring Boot microservices were designed stateless (session in Redis, no local state) to enable horizontal scaling. We ran multiple instances behind a load balancer. For DB, we had read replicas to scale read traffic horizontally.

Q71. [Behavioral] Describe a time you had to debug a difficult production issue.

Structure as STAR. Example: Situation — production API returning 503s intermittently after deployment. Task — identify and fix root cause urgently. Action — checked logs: saw 'HikariPool-1 - Connection is not available, request timed out after 30000ms.' DB connections exhausted. Ran SHOW PROCESSLIST — saw many queries running 10+ seconds on a new endpoint. Found: a JPA query without pagination loading 50,000 records into memory on every call. Fixed: added pagination (limit 100), added an index on the ORDER BY column. Result — connection pool healthy, 503s stopped, query time dropped from 10s to 50ms.

Q72. [Behavioral] How do you keep up with new technologies in the Java/backend ecosystem?

Be genuine: 'I follow [specific sources — Spring Blog, InfoQ, Baeldung, High Scalability blog, YouTube: Amigoscode, Tech with Tim]. I try to build small proof-of-concept projects when learning something new — for example, when learning Kafka I built a small order event pipeline locally to understand partitions and consumer groups hands-on. At Deepta AI I took initiative to learn and implement a RAG pipeline end-to-end, which gave me practical experience with vector databases and LLM integration beyond my core Java background.'

Q73. [Behavioral] Tell me about a time you improved the performance of a system.

STAR format. Quantify the improvement. Example: 'Our report generation API was timing out for clients with large datasets. I profiled it with EXPLAIN ANALYZE and found it was loading all records into memory and filtering in Java. I rewrote it to push filtering and pagination to the DB layer, added a composite index on (tenant_id, created_at), and changed the entity loading to use a projection DTO (SELECT only needed columns instead of *). Report generation time dropped from 25 seconds to 400ms for 90th percentile requests, and timeout errors dropped to zero.'

Q74. [Behavioral] How do you handle disagreements with teammates on technical decisions?

'I believe good technical decisions come from evidence, not authority. When I have a different view, I try to articulate the trade-offs clearly — ideally with data: benchmark numbers, failure scenarios, or prior art. If the disagreement is significant, I suggest a time-boxed proof of concept to compare options empirically. If it's a close call and we're stuck, I defer to the team consensus or tech lead's call and commit fully — partial commitment causes more damage than the 'wrong' technical choice. I document the decision and its rationale either way.'

Q75. [Behavioral] Where do you see yourself in 3-5 years?

'I want to deepen my expertise in distributed systems and backend architecture — move from building features to designing scalable, reliable systems. Over the next 2-3 years, I want to work on challenging backend problems at scale, gaining experience with high-throughput Kafka pipelines, multi-region systems, and performance engineering. Longer term, I'm interested in a senior/lead engineer track where I can both architect systems and mentor junior engineers. This role at Infosys aligns well because [mention specific team/project if you know it].'

Section H: Quick Reference — Q76–Q100

These are commonly asked short-answer questions. Prepare 2-3 sentence answers for each.

GOOD TO KNOW

Q76. What is SOLID in object-oriented design?

Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. Know a one-sentence definition and one real example for each principle. Most asked: Single Responsibility (one class, one reason to change) and Dependency Inversion (depend on abstractions not concretions — the foundation of Spring DI).

Q77. What is the difference between EAGER and LAZY loading in JPA?

EAGER: associated entity loaded immediately with parent query. LAZY: loaded only when the field is first accessed (proxy object returned initially). Most @OneToMany should be LAZY to avoid loading large collections unnecessarily. @ManyToOne can be EAGER if you always need it. Watch for LazyInitializationException when accessing LAZY field outside a transaction (open session in view anti-pattern).

Q78. What is a DTO and why use it?

Data Transfer Object — plain class for transferring data between layers (controller ↔ service ↔ client). Decouples API contract from domain model and DB schema. Prevents exposing sensitive fields, avoids Jackson serialization issues with JPA bidirectional relationships, and lets you evolve DB schema without breaking API clients. Map entities to DTOs using MapStruct (compile-time code generation) or ModelMapper.

Q79. What is database connection pooling and why is it needed?

Creating a new DB connection per request is expensive (TCP handshake, auth, ~50-200ms). A connection pool maintains a set of reusable pre-established connections. Threads borrow a connection, use it, and return it to the pool. HikariCP is Spring Boot's default — configure `maximumPoolSize` based on DB capacity and thread count. Too large: exhausts DB's `max_connections`; too small: threads wait.

Q80. What is idempotency, and why does it matter for REST APIs and Kafka?

An idempotent operation produces the same result whether called once or multiple times. GET, PUT, DELETE are idempotent by definition. POST is NOT idempotent by default. For safe retries, add idempotency keys to POST APIs (client sends unique key in header; server stores processed keys and returns cached result for duplicate). In Kafka: idempotent consumers re-process the same message safely (UPSERT instead of INSERT, check if already processed).

Q81. What is the difference between @Entity and @Table annotations in JPA?

@Entity: marks the class as a JPA managed entity (maps to a DB table). **@Table**: optional, specifies the actual table name, schema, and constraints. If **@Table** is omitted, JPA uses the class name as the table name. **@Column**: customizes individual field mapping (column name, nullable, unique, length). **@Id**: marks primary key field. **@GeneratedValue**: specifies PK generation strategy (IDENTITY: DB auto-increment, SEQUENCE: DB sequence, AUTO: JPA decides).

Q82. What is CORS and how do you handle it in Spring Boot?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks frontend JavaScript from making HTTP requests to a different origin (domain/port/protocol) than the page it loaded from. In Spring Boot: configure globally with **@Bean CorsConfigurationSource** or per-controller with **@CrossOrigin** annotation. In Spring

Security filter chain, add `.cors(cors -> cors.configurationSource(corsConfigSource))` to apply CORS before security filters. For production, restrict allowedOrigins to your actual frontend domains — never use '*' in production with credentials.

Q83. What is the difference between monolithic and microservices architecture in terms of deployment?

Monolith: one deployable unit (JAR/WAR) — simple deployment but any change requires full rebuild/redeploy, scales as one unit. **Microservices:** each service deployed independently — a fix in OrderService deploys only OrderService without touching PaymentService. Enables polyglot deployment (different languages, frameworks), independent scaling, smaller blast radius of bugs. Trade-off: more complex infrastructure (container orchestration, service discovery, distributed tracing).

Q84. What is Spring Boot's @Profile annotation and how do you use it?

@Profile activates a bean or configuration class only when a specific Spring profile is active (e.g., dev, test, prod). Use case: different DataSource beans for test (H2 in-memory) vs prod (PostgreSQL), different security configurations, mock vs real third-party service implementations. Set active profile via: `spring.profiles.active=prod` in `application.yml`, `SPRING_PROFILES_ACTIVE` environment variable, or `-Dspring.profiles.active=prod` JVM argument.

Q85. What is Hibernate's first-level cache vs second-level cache?

First-level (L1) cache: per-EntityManager session cache — automatic in JPA. Within one transaction, fetching the same entity twice returns the same instance without hitting DB. Cleared when session closes. **Second-level (L2) cache:** shared across EntityManagers — optional, typically Ehcache or Hazelcast. Caches entities by PK across transactions. Use carefully: invalidation complexity, stale data risk. For most Spring Boot apps, Redis @Cacheable at service layer is simpler than JPA L2 cache.

Q86. How do you implement soft delete in Spring Boot JPA?

Soft delete marks records as deleted (`is_deleted = true` or `deleted_at = timestamp`) without physically removing the row. Implementation: add @Column Boolean `isDeleted` field to entity; override delete logic in service to set `isDeleted = true` + `save`. To exclude soft-deleted records from all queries, use @SQLWhere(`clause = 'is_deleted = false'`) on the entity (Hibernate-specific) or a JPA @EntityListeners-based approach. Alternatively, use Spring Data's filtering in repository custom queries.

Q87. What is WebSocket and when would you use it over REST?

WebSocket is a full-duplex, persistent TCP connection between client and server — both can send messages at any time after the initial HTTP upgrade handshake. Use over REST for: real-time features (live chat, collaborative editing, live price feeds, gaming, live notifications). REST is request-response (client pulls); WebSocket is push-capable (server pushes to client). In Spring Boot: STOMP over WebSocket with `@EnableWebSocketMessageBroker` for pub/sub messaging with topic subscriptions. Trade-off: WebSocket connections are stateful and more complex to scale horizontally (need sticky sessions or a message broker like Redis Pub/Sub to relay messages across instances).

Q88. What is `@Scheduled` in Spring Boot and what are its limitations?

`@Scheduled(cron = '0 0 * * * *')` runs a method on a schedule within the Spring context. Simple for single-instance jobs. Limitations: runs in every instance in a multi-node deployment (so a job runs N times, once per instance — may cause duplicate processing). Solution: use `@ScheduledLock` (ShedLock library) — acquires a distributed lock in DB/Redis before executing, ensuring only one instance runs the job at a time. For complex scheduling/job management: use dedicated schedulers like Quartz (persistent job store, clustering) or cloud-native solutions like Kubernetes CronJob.

Q89. How would you implement health checks in a Spring Boot microservice?

Spring Boot Actuator provides `/actuator/health` endpoint out of the box. It aggregates health from all registered HealthIndicators: `DataSourceHealthIndicator` (DB connectivity), `RedisHealthIndicator`, `KafkaHealthIndicator`, `DiskSpaceHealthIndicator`, and custom ones you write. Configure Kubernetes liveness probe (process alive?) and readiness probe (ready to serve traffic?) to use `/actuator/health` or separate `/actuator/health/liveness` and `/actuator/health/readiness` groups. Readiness probe failing causes load balancer to stop sending traffic to that instance without killing it.

Q90. Explain the purpose of `@Transient` in JPA.

`@Transient` marks a field that should NOT be persisted to the database — it's excluded from JPA's column mapping. Used for: computed fields (derived from persisted fields, no need to store), temporary/helper fields used in business logic, and fields that are populated by the application (not from DB). Java's transient keyword has a similar effect in JPA as well, but `@Transient` is more explicit and IDE-friendly.

Q91. What is the difference between `CascadeType.MERGE`, `PERSIST`, and `ALL` in JPA?

Cascade controls which entity lifecycle operations propagate from parent to child. **PERSIST**: saving parent also saves child. **MERGE**: updating parent also merges child. **REMOVE**: deleting parent also deletes child (use carefully — unintentional deletes). **REFRESH**: refreshing parent refreshes child. **ALL**: applies all cascade types. Common combination: `@OneToMany(cascade = {PERSIST, MERGE}, orphanRemoval = true)` — saving/updating the parent manages children, and children removed from the collection are deleted.

Q92. What is the n-tier architecture and how does it apply to Spring Boot?

N-tier (typically 3-tier for web apps) separates concerns into layers: **Presentation** (Controller — `@RestController`, handles HTTP), **Business Logic** (Service — `@Service`, orchestrates domain logic), **Data Access** (Repository — `@Repository`, DB queries via JPA). Each layer depends only on the layer below it (Controller → Service → Repository), never skipping layers or going upward. This enables independent testing (mock service in controller test, mock repository in service test) and separation of concerns.

Q93. What is the Command pattern and how does it relate to any of your project work?

Command pattern encapsulates a request as an object (Command), separating the sender from the executor. Enables: queuing commands, undo/redo, audit logging. In practice: CQRS (Command Query Responsibility Segregation) is an architectural application of this — separate command objects for writes (CreateOrderCommand) from queries for reads. In Kafka: each Kafka message can be seen as a Command/Event. Spring's `@Async` with a task object follows a similar pattern. Know the concept and one real connection to your work.

Q94. What's the difference between optimistic and pessimistic locking in a concurrent scenario?

Pessimistic locking: lock the record in DB when you read it (`SELECT ... FOR UPDATE`) — prevents other transactions from reading/writing. Good for high contention, short transactions. Can cause deadlocks. **Optimistic locking** (`@Version` in JPA): no lock on read. On update, check that the version hasn't changed since you read it — if it has, another transaction modified it, throw `OptimisticLockException` and retry. Good for low contention (most updates succeed). Spring Data JPA: add `@Version` Long version field to entity — Spring handles the version check automatically.

Q95. How do you handle large file uploads in a Spring Boot API?

Strategy: (1) For files that can fit in memory (<10MB): `@RequestParam MultipartFile` file in controller, process directly. (2) For large files: use presigned S3 URL approach — client requests a presigned PUT URL from backend, uploads directly to S3 without going through your server, backend receives S3 event notification when upload completes. Avoids memory pressure and bandwidth on your app servers. (3) Increase Spring's multipart size limits if needed (`spring.servlet.multipart.max-file-size`). (4) Stream processing for very large files: `InputStream` from `MultipartFile`, process in chunks without loading everything into memory.

Q96. What is eventual consistency and when is it acceptable?

Eventual consistency means that, given no new updates, a distributed system will eventually converge to the same state across all nodes — but at any given moment, different nodes may temporarily return different values. Acceptable when: slight staleness is tolerable (product catalog views, social media counts, recommendation results). NOT acceptable when: strong consistency is required (bank balances, inventory counts preventing overselling, authentication state). In microservices with Kafka: a service's DB might not reflect another service's event immediately — design consumers to handle arriving events idempotently and handle 'not yet consistent' states gracefully.

Q97. What is Docker and how do you use it with Spring Boot?

Docker packages an application and all its dependencies (JRE, config, native libs) into a portable container image that runs consistently on any host. Spring Boot app → Dockerfile: `FROM eclipse-temurin:17-jre, COPY app.jar app.jar, ENTRYPOINT ['java', '-jar', 'app.jar']`. Use multi-stage builds to keep final image small (compile in a JDK image, run in a JRE image). Spring Boot 2.3+ has built-in Buildpacks support: `./mvnw spring-boot:build-image` creates an optimized OCI image without a Dockerfile. Run locally: `docker run -p 8080:8080 -e SPRING_DATASOURCE_URL=... myapp:latest`.

Q98. What is Kubernetes and how does it relate to your microservices?

Kubernetes (K8s) is a container orchestration platform that automates: deployment, scaling, self-healing (restarts failed pods), service discovery (ClusterIP service + DNS), config management (ConfigMaps, Secrets), and rolling updates/rollbacks. For Spring Boot microservices: each service runs as a Deployment (N replicas of a pod), exposed via a Service (stable DNS), configured via ConfigMap (non-sensitive) and Secret (sensitive). Horizontal Pod Autoscaler scales pods based on CPU/memory. Spring Boot Actuator health endpoints integrate with K8s liveness and readiness probes.

Q99. How do you approach writing code that's easy to test?

Key practices: (1) Dependency injection over direct instantiation — inject dependencies via constructor so tests can provide mocks. (2) Single Responsibility — small, focused methods are easy to test in isolation. (3) Avoid static method calls for things you need to mock — use injectable services instead. (4) Avoid global state / thread-local state — hard to isolate in tests. (5) Write tests first (TDD) or immediately after implementing — forces testable design. (6) Use meaningful test names: 'shouldReturnNotFoundWhenEmployeeDoesNotExist' tells you what's being tested and expected. (7) Follow Arrange-Act-Assert pattern clearly.

Q100. What questions do you have for us?

Always have 2-3 thoughtful questions: (1) 'What does the tech stack look like for the team I'd be joining, and what are the main backend challenges you're working on right now?' (2) 'How does your team approach technical debt — is there dedicated time for refactoring and improving existing systems?' (3) 'What does the onboarding process look like for a new engineer, and how quickly can someone expect to contribute to production code?' These show you're serious, technically curious, and thinking about the long term.

End of Part 10 — Resume Deep Dive (100 Questions)